SKALA 4기 · agile 방법론과 MSA · 팀 프로젝트

SK AM 프로젝트 기획서

SK API Marketplace

기업용 AI 컴포넌트 원스톱 조달 플랫폼

소속	SKALA 판교캠퍼스 9반
팀명	4조 (블루베리스무디)
팀원	김민정(P282) · 김지수(P285) · 박규리(P289) · 오승민(P293) · 전경호(P303)
문서 종류	Agile / MSA 팀 프로젝트 제출용 기획서
작성일	2026년 8월 28일
버전	1.0

> 본 기획서는 발표자료(PPT)와 별도로, 프로젝트의 목적, 스프린트 구분, 주제 선정 배경, 이해관계자 분석을 중심으로 작성되었다.

목 차

I. 프로젝트 개요
1.1 서비스 소개 1.2 팀 구성 및 역할

II. 주제 선정 배경
2.1 시장 현황 2.2 도메인 매칭

III. 이해관계자 정의 및 Pain Point
3.1 구매 기업 3.2 API 공급사 3.3 플랫폼
3.4 데이터로 보는 Pain Point 3.5 경쟁 벤치마킹

IV. 솔루션
4.1 핵심 솔루션 4.2 AI 차별화 기능

V. Product Backlog
5.1 Story Point 산정 기준 5.2 우선순위 판단 기준

VI. 스프린트 구분
6.1 Sprint 1 6.2 Sprint 2 6.3 분할 근거 6.4 Sprint Backlog

VII. Task 분해
7.1 Sprint 1 Task 7.2 Sprint 2 Task

VIII. 협업 운영 방식
8.1 노션 기반 협업 8.2 Kanban 8.3 Daily Scrum
8.4 형상 관리 8.5 트러블슈팅 기록

IX. 화면 설계
9.1 메인 9.2 로그인·회원가입 9.3 API 목록
9.4 구독자(BUYER) 관점 9.5 제공사(PROVIDER) 관점

X. 아키텍처 개요

XI. API 명세 설계 (부록 A)
A-0 공통 규약 · A-1~5 도메인 매핑 · A-6~7 주요 API 명세 · A-8 구독 흐름

I. 프로젝트 개요

1.1 서비스 소개

[표 1] 서비스 개요

서비스명	SK AM (SK API Marketplace)
한 줄 소개	SK 계열사 및 파트너사의 AI 엔진을 낱개 API 상품으로 등록 및 구독할 수 있는 B2B AI API 마켓플레이스
서비스 형태	B2B

실습 템플릿(온라인 교육 플랫폼: 강사, 과목, 수강신청, 결제)의 구조를 그대로 재활용하고, 개념만 AI API 마켓플레이스 도메인으로 치환하여 설계했다.

1.2 팀 구성 및 역할

[표 2] 팀 구성 및 역할

이름	번호	역할	담당 업무
김민정	P282	PO / 프론트엔드	제품 방향 총괄, 화면 구현
김지수	P285	SM / 기획	스크럼 마스터, 자료 조사 및 발표자료 총괄
박규리	P289	백엔드	도메인 변경 및 API 명세 설계
오승민	P293	프론트엔드	UI 코드 및 화면 구현
전경호	P303	발표 / 기획	최종 발표 및 기획 지원

II. 주제 선정 배경

2.1 시장 현황

현재 글로벌 AI API 시장은 가파르게 성장하고 있다.

[표 3] AI API 시장 현황 및 SK 자산 현황

구분	내용
CAGR	약 32%로, 일반 IT 시장 평균보다 3배 빠른 성장세를 보인다.
검증된 모델	SK Open API로 TMAP, 지오비전 등 자사 기술을 이미 API로 상품화하여 판매 중이다.
미연결 자산	SKT LLM, 하이닉스 비전 AI 등 뛰어난 계열사 자산이 정부사업 및 지원 프로그램 단위로만 제공되어 상시 연결되지 않는다.

> 출처: Fortune Business Insights, AI API Market Report, 2026.08

결론. 검증된 공급(계열사 AI 자산)과 빠르게 성장하는 수요(AI API 시장) 사이를 잇는 통합 창구의 부재가 곧 기회이다. 이미 있는 서비스와 자산을 최대한 재활용하면서, 이를 하나의 신뢰 가능한 플랫폼으로 묶는 것이 이번 프로젝트의 출발점이다.

2.2 도메인 매칭

템플릿(온라인 교육 플랫폼)의 개념 구조를 그대로 유지하면서, 도메인만 AI API 마켓플레이스로 치환했다.

[표 4] 템플릿 개념과 SK AM 개념의 대응

템플릿 개념	SK AM
강사 / 학생	API Vendor / 구매 기업 담당자
과목 등록	API 상품 등록 (카테고리, 요금, 스펙)
과목 조회	API 카탈로그 탐색
수강신청	API 구독 신청
결제	구독 결제
수강권한 노출	구독 상태(ACTIVE)로 API 호출 권한 표현
수강생 수	구독 기업 수 (인기도 지표)
추천	사용 패턴 기반 API 추천

III. 이해관계자 정의 및 Pain Point

본 장에서는 팀이 정의한 이해관계자(구매 기업, API 공급사, 플랫폼 운영자)가 실제로 겪는 불편함이 무엇인지, 왜 그 문제가 중요한지를 정의한다.

3.1 구매 기업 (Consumer)

> "AX, 전부 도입하기에는 너무 부담스러워요."

현황 및 문제점

- 중소기업은 AX가 필요하다는 것은 알지만, 데이터센터, MLOps 인프라, 전담 AI 인력을 갖춘 풀스택 AX 솔루션을 통째로 도입하기에는 초기 투자비와 유지보수비, 학습곡선이 모두 부담스럽다.
- 대기업 대상 엔터프라이즈 AX 패키지는 전사 계약과 장기 라이선스 구조여서 필요한 기능만 소량으로 시험해보기 어렵다.
- 종량제(pay-per-use)는 유연해 보이지만, 사용량이 튀면 예상치 못한 과금이 발생해 예산 관리가 어려운 중소기업에는 오히려 재무 리스크로 작동한다.

해결 방향: API 단위 구독제(월정액) 구매

- 전체 AX를 사지 않고, 지금 당장 필요한 기능(OCR, 요약, 챗봇 등)만 API 단위로 골라 월 구독 플랜으로 도입한다. 매달 나가는 비용이 고정되어 예산 계획이 쉬워진다.
- 초기 CAPEX 없이 정액 OPEX로 전환하여 자금 리스크를 최소화하면서, 종량제 대비 비용 예측 불가능성까지 해소한다.
- 사업이 커지면 상위 플랜으로 업그레이드하거나 다른 API 구독을 추가하며 점진적으로 AX 범위를 확장할 수 있다(업셀 여지).

3.2 API 공급사 (Producer)

> "기술은 있지만 수익화 채널이 불안정해요."

현황 및 문제점

- SK AX 및 계열사(SKT, SK하이닉스 등)는 자체 AI 역량과 데이터를 보유하고 있으나, 활용 방식이 대형 엔터프라이즈 단위 B2B 계약에 편중되어 있다.
- 대형 계약만 좇으면 소규모, 다품종 수요는 놓치고, 종량제 기반 수익은 사용량 변동에 따라 매출이 들쭉날쭉해 예측이 어렵다.

해결 방향: 구독제 기반 안정적 반복 수익(MRR) 및 ESG 가치

- 계열사별 AI 엔진을 API 단위 구독 상품으로 쪼개 마켓플레이스에 등록하면, 다수 중소기업의 월 구독료가 모여 안정적인 반복 매출(MRR) 구조를 확보할 수 있다.
- 구독제는 종량제보다 수요 예측과 인프라 캐파 플래닝이 쉬워, 서버 및 모델 운영 계획을 안정적으로 세울 수 있다.

- 동시에 AX 인프라가 부족한 중소기업의 디지털 전환 진입장벽을 낮춘다는 점에서 ESG(사회적 가치) 경영 성과로도 연결된다.

3.3 플랫폼 (SK AX) 차원

> "신뢰 가능한 통합 창구가 없습니다."

현황 및 문제점

- 계열사마다 AI 서비스를 각자 흩어진 방식으로 제공하면, 수요 기업 입장에서는 탐색 비용이 크고 계열사 간 API 품질, 문서, 인증, 과금 방식도 제각각일 위험이 있다.
- 특정 스타트업이 운영하는 마켓플레이스는 보안, 안정성, SLA에 대한 신뢰도가 낮아, 구독처럼 지속적 관계를 맺어야 하는 계약을 기업 고객이 주저하게 된다.

해결 방향: SK AX 단일 신뢰 플랫폼 및 계열사 개방형 등록 구조

- SK AX가 플랫폼 운영 주체(인증, 구독 결제 및 과금, SLA, 대시보드)를 맡아 대기업 수준의 신뢰성을 담보한다.
- SKT, SK하이닉스 등 계열사가 각자의 AI 엔진을 이 플랫폼에 구독 상품으로 등록할 수 있게 개방하여, 하나의 창구에서 다양한 도메인의 검증된 API를 만날 수 있게 한다.

3.4 데이터로 보는 Pain Point

[표 5] Pain Point 관련 시장 지표

지표	내용
기업당 평균 API 보유 수	2년 전 약 200개에서 2026년 354개로 증가
API 관리 시장 규모	2025년 52.4억 달러에서 2026년 69.2억 달러로 성장 (연 32%)
계약 갱신 비용	일부 벤더가 사전 고지 없이 AI 기능을 추가하면서 계약 갱신 비용이 최대 30%까지 증가 (가트너)
거버넌스 공백	기업의 80%가 AI를 사용 중이나, 자사의 AI 활용 사례를 완전히 파악 및 문서화한 곳은 21%에 불과

3.5 경쟁 벤치마킹: SK Open AI

SK는 이미 SK Open AI로 자사 기술(TMAP, 지오비전 등)을 API로 공개하고 있다. 창구가 여러 곳으로 파편화되어 있지는 않지만, 다음 세 가지 점에서 SK AM이 겨냥할 틈새가 있다고 봤다.

[표 6] SK Open AI와 SK AM의 지향점 차이

관점	SK Open AI (현재)	SK AM (지향)
플랫폼 성격	위치 기반, 데이터 등 다양한 자사 기술이 섞인 범용 Open API 창구. AI에 특화된 플랫폼은 아니다.	AI 기능(비전·음성·언어·문서 등)만 모은 전용 마켓플레이스
구매 방식	개별 API를 건별로 신청·연동. "필요한 기능을 골라 묶어서 구독"하는 형태는 아니다.	카테고리로 탐색하고 API 단위로 월 정액 구독
중소기업 친화성	요금·문서·온보딩이 개발 역량을 갖춘 사용자를 전제로 한다.	요금표 투명 공개, 카탈로그·추천·구독 현황을 한 화면에서 제공

착안점. SK Open AI는 "SK 기술을 API로 판다"는 방향은 같지만, AI 전용 플랫폼도 아니고 중소기업이 바로 골라 쓰기에 친화적이지도 않다. SK AM은 이미 검증된 계열사 AI 자산을 **기능 단위로 골라 구독하는 마켓플레이스** 형태로 다시 묶어, 이 틈새를 메우는 것을 목표로 한다.

IV. 솔루션

4.1 핵심 솔루션: 필요한 API만 구독하는 구조

[표 7] 이해관계자별 Before / After

이해관계자	Before (Pain Point)	After (Solution)
구매 기업	전체 AX 솔루션의 초기비용 부담, 종량제의 예산 관리 어려움	필요한 API만 선택해 구독. 매달 고정 비용으로 예산 관리 가능
API 공급사	대형 B2B 계약에만 의존, 소규모 수요를 놓치고 매출도 불안정	여러 중소기업의 구독료가 모여 안정적인 반복 매출(MRR) 형성

4.2 AI 차별화 기능

추천 (Recommend) — 현재 구현 범위

구독 이력을 기반으로 AI가 다음 API를 추천한다. 구독한 API와 같은 카테고리를 찾고, 그중 가장 많은 기업이 쓰는 인기 API 순으로 추천하는 방식이다.

향후 확장 가능한 기능

- **자연어 검색.** "한국어 요약 API 찾아줘"와 같이 조건을 말하면 적합한 API를 필터링한다.
- **성능 비교.** 후보로 고른 API들에 같은 샘플 데이터를 넣어 속도와 정확도를 자동으로 비교한다.

V. Product Backlog User Story · Story Point · Priority

Product Backlog는 이해관계자별 User Story를 기준으로 작성했으며, Story Point는 Fibonacci(1, 2, 3, 5, 8) 기준의 팀 내 상대적 추정치이다.

[표 8] Product Backlog

ID	User Story	SP	우선순위	Sprint
US-01	구매 기업 담당자로서, 이메일과 비밀번호로 회원가입하고 로그인할 수 있다. 그래야 내 계정으로 API를 구독하고 관리할 수 있다.	3	High	Sprint 1
US-02	사용자로서, 회원가입 시 역할(구매 기업 담당자 / API Vendor)을 선택할 수 있다. 그래야 내 권한에 맞는 화면과 기능을 이용할 수 있다.	2	High	Sprint 1
US-03	사용자로서, 로그아웃하여 세션을 안전하게 종료할 수 있다.	1	High	Sprint 1
US-04	API Vendor로서, 내가 제공하는 AI API를 카테고리, 요금, 스펙과 함께 상품으로 등록할 수 있다. 그래야 구매 기업이 내 API를 찾아볼 수 있다.	5	High	Sprint 1
US-05	구매 기업 담당자로서, 등록된 API 카탈로그를 조회하고 상세 스펙을 확인할 수 있다. 그래야 우리 회사에 필요한 API를 비교할 수 있다.	5	High	Sprint 1
US-06	구매 기업 담당자로서, 마음에 드는 API를 구독 신청할 수 있다. 그래야 해당 API를 우리 서비스에 연동할 수 있다.	5	High	Sprint 1
US-07	구매 기업 담당자로서, 내가 구독 중인 API 목록을 한눈에 확인할 수 있다. 그래야 현재 이용 중인 API 현황을 관리할 수 있다.	3	High	Sprint 1
Sprint 2 — 확장				
US-08	구매 기업 담당자로서, 카테고리(비전, 음성, 언어, 문서, 생성형, 예측, 이상탐지 등)로 API를 필터링해서 찾아볼 수 있다.	3	Medium	Sprint 2
US-09	구매 기업 담당자로서, 마이페이지에서 내 구독 이력 기반 추천 API를 확인할 수 있다. 그래야 같은 카테고리의 인기 API를 놓치지 않고 도입할 수 있다.	5	Medium	Sprint 2

ID	User Story	SP	우선순위	Sprint
US-10	구매 기업 담당자로서, 구독한 API에 대해 월 정액 결제를 진행할 수 있다. 그래야 지속적으로 API를 이용할 수 있다.	5	High	Sprint 2
US-11	구매 기업 담당자로서, 결제 및 크레딧 사용 내역을 조회할 수 있다. 그래야 우리 회사의 API 이용 비용을 관리할 수 있다.	3	Medium	Sprint 2
US-12	API Vendor로서, 내 API의 구독 기업 수(인기도 지표)를 확인할 수 있다. 그래야 시장 반응을 파악할 수 있다.	2	Low	Sprint 2
확장 백로그				
US-13	구매 기업 담당자로서, 자연어로 조건을 말하면 적합한 API를 추천받을 수 있다. 그래야 카탈로그를 일일이 뒤지지 않아도 된다.	8	Low	확장
US-14	구매 기업 담당자로서, 후보 API들에 같은 샘플 데이터를 넣어 속도와 정확도를 자동으로 비교해볼 수 있다.	8	Low	확장
합계 (14개 스토리)		**58**		

> 전체 Story Point 합계는 58점이며, Sprint 1이 24점, Sprint 2가 18점, 확장 백로그가 16점이다. 실제 진행 속도(Velocity)에 따라 조정이 필요하다.

5.1 Story Point 산정 기준

소요 시간이 아니라 팀 내 상대적인 크기로 매겼다. 기준점은 US-01 회원가입 및 로그인을 3점으로 두고, 나머지를 여기에 비교해 Fibonacci 수열로 붙였다. 점수를 키우는 요인은 연동 서비스 수, 비동기 처리 여부, 팀이 처음 다뤄보는 정도 세 가지이다.

[표 9] Story Point 산정 기준

SP	기준	해당 스토리
1	기존 화면에 동작 하나를 붙이는 수준	US-03
2	화면에 필드나 지표를 추가하는 수준으로, 새 API 호출이 거의 없음	US-02, US-12
3	조회 API 한 번과 화면 하나 (기준점)	US-01, US-07, US-08, US-11
5	쓰기 요청에 검증 및 권한 분기가 붙거나, 서비스 두 개 이상이 연동됨	US-04, US-05, US-06, US-09, US-10
8	설계부터 새로 해야 하며, 이번 일정에 그대로 넣을 수 없는 크기	US-13, US-14

5.2 우선순위 판단 기준

[표 10] 우선순위 판단 기준

등급	판단 기준	해당 스토리
High	회원가입에서 구독까지 이어지는 핵심 흐름 위에 있거나, 서비스의 가치 제안(월 정액)을 직접 증명하는 기능	US-01 ~ US-07, US-10
Medium	사용자 경험을 개선하지만 없어도 핵심 흐름은 성립하는 기능	US-08, US-09, US-11
Low	부가 지표이거나, 신규 설계가 필요해 이번 기간에 담기 어려운 기능	US-12, US-13, US-14

VI. 스프린트 구분

정의한 솔루션 중 핵심 가치를 검증할 기능은 Sprint 1(MVP)에서 먼저 구현하고, 부가가치 기능은 Sprint 2(확장)로 미뤘다.

6.1 Sprint 1 (MVP)

핵심 흐름은 회원가입에서 API 등록, 카탈로그 조회를 거쳐 구독 신청에 이르는 경로이다. 포함 기능은 다음과 같다.

- 로그인, 로그아웃, 회원가입
- API 조회 및 등록
- 구독
- 내 구독

6.2 Sprint 2 (확장)

확장 흐름은 구독 결제와 AI 추천으로 이어지며, 시간이 허용될 경우 자연어 검색과 성능 비교까지 확장한다. 포함 기능은 다음과 같다.

- 카테고리 필터
- 마이페이지 (추천)
- 결제

6.3 스프린트를 이렇게 나눈 이유

1) Sprint 1에서는 회원가입부터 구독까지의 최소 핵심 흐름만 갖춰, 서비스의 가치(필요한 API를 찾아 구독한다)를 가장 빠르게 검증하는 것을 목표로 했다.
2) 결제, 추천 등 부가가치 기능은 Sprint 2로 미뤄, 핵심 플로우가 안정된 뒤에 수정 및 확장하기 쉬운 구조를 유지했다.
3) 이런 스프린트 구분을 통해 이미 완성되어 있는 MSA 아키텍처(코드 템플릿)를 새로운 도메인과 서비스 목적에 맞게 점진적으로 수정 및 연동하며, 분산 시스템 구조를 단계적으로 이해하는 것을 함께 목표로 삼았다.

6.4 Sprint Backlog

각 스프린트에 담은 User Story를 Story Point(SP)와 우선순위(P)와 함께 정리하면 다음과 같다. Product Backlog([표 8])에서 Sprint 1·2에 배정된 항목만 추려낸 것이다.

[표 11] Sprint Backlog (US + SP + P)

구분	ID	User Story (요약)	SP	P
Sprint 1 — 회원가입부터 구독 신청까지 핵심 흐름 한 바퀴 (2일차 오전 ~ 13시)				
S1	US-01	이메일·비밀번호로 회원가입·로그인	3	High
S1	US-02	회원가입 시 역할(구매 기업 / API Vendor) 선택	2	High
S1	US-03	로그아웃으로 세션 종료	1	High
S1	US-04	API 상품 등록 (카테고리·요금·스펙)	5	High
S1	US-05	API 카탈로그 조회 및 상세 스펙 확인	5	High
S1	US-06	API 구독 신청	5	High
S1	US-07	내 구독 API 목록 확인	3	High
Sprint 1 계획 SP			**24**	
Sprint 2 — 결제·탐색 기능 확장 (2일차 13 ~ 15시)				
S2	US-08	카테고리로 API 필터링	3	Medium
S2	US-09	마이페이지 구독 이력 기반 추천 확인	5	Medium
S2	US-10	구독 API 월 정액 결제	5	High
S2	US-11	결제·크레딧 사용 내역 조회	3	Medium
S2	US-12	내 API의 구독 기업 수 확인	2	Low
Sprint 2 계획 SP			**18**	

> Sprint 1 목표: 신규 계정으로 가입해 API를 등록하고, 카탈로그에서 찾아 구독하면 내 구독 목록에 나타난다.
> Sprint 2 목표: 구독 시 결제가 기록되어 조회되고, 카테고리로 카탈로그를 좁힐 수 있으며, 추천이 노출된다.

[표 12] 스프린트별 진행 현황

구분	계획 SP	완료 SP	달성률	미완 항목
Sprint 1	24	24	100%	없음
Sprint 2	18	13	72%	US-09 마이페이지 추천(5)
합계	**42**	**37**	**88%**	**5점 이월**

Sprint 2에서 미완으로 남은 항목은 US-09 한 건이다. 추천은 카탈로그 상단 노출까지 구현했으나 마이페이지의 구독 이력 기반 추천까지는 닿지 못했다. 이 건은 다음 스프린트의 첫 작업으로 올린다.

VII. Task 분해

User Story를 실제 작업 단위로 나눌 때, 한 사람이 맡아 끝내고 다음 사람에게 넘길 수 있는 크기를 기준으로 삼았다. 아래는 칸반 보드에 등록해 운영한 실제 Task 목록이다. 각 Task에는 담당 영역(Backend, Frontend, Infra, 기획·문서)과 우선순위(P0 필수, P1 중요, P2 여유되면) 태그를 달았다.

7.1 Sprint 1 Task

[표 13] Sprint 1 Task 목록

흐름 단계	Task	담당 영역	우선순위
API 등록	catalog(course) 도메인 치환: 엔티티, enum, 테이블, 경로	Backend	P0 필수
	Category enum 8종 교체 (VISION / SPEECH / LANGUAGE / DOCUMENT / GENERATIVE / FORECAST / ANOMALY / OTHER)	Backend	P0 필수
	ApiProduct.Status 교체 (ACTIVE / INACTIVE → PUBLISHED / DEPRECATED)	Backend	P1 중요
	API 등록 화면의 카테고리 option 목록 교체	Frontend	P0 필수
회원가입	User.Role 교체 (STUDENT / INSTRUCTOR → BUYER / PROVIDER)	Backend	P0 필수
	isInstructor를 isProvider로 전면 치환 (6곳)	Frontend	P0 필수
	회원가입 화면의 역할 선택 문구 변경	Frontend	P0 필수
카탈로그 조회	프론트 store 치환 및 categoryLabelMap 8종 교체	Frontend	P0 필수
구독 신청	subscription(enrollment) 도메인 치환 및 courseId를 productId로 변경	Backend	P0 필수
공통	라우터 경로 치환 (/courses → /products, /enrollments → /subscriptions)	Frontend	P0 필수
	프론트 API 모듈 치환 (course.js → product.js, enrollment.js → subscription.js)	Frontend	P0 필수
	API Gateway 라우팅 규칙 수정	Infra	P0 필수
	브랜딩 및 문구 교체 (LearnNexus → 팀 서비스명, 로고, 랜딩 카피)	Frontend	P1 중요
	Sprint 1 통합 검증: 가입, 등록, 조회, 구독신청 한 바퀴	기획·문서	P0 필수

7.2 Sprint 2 Task

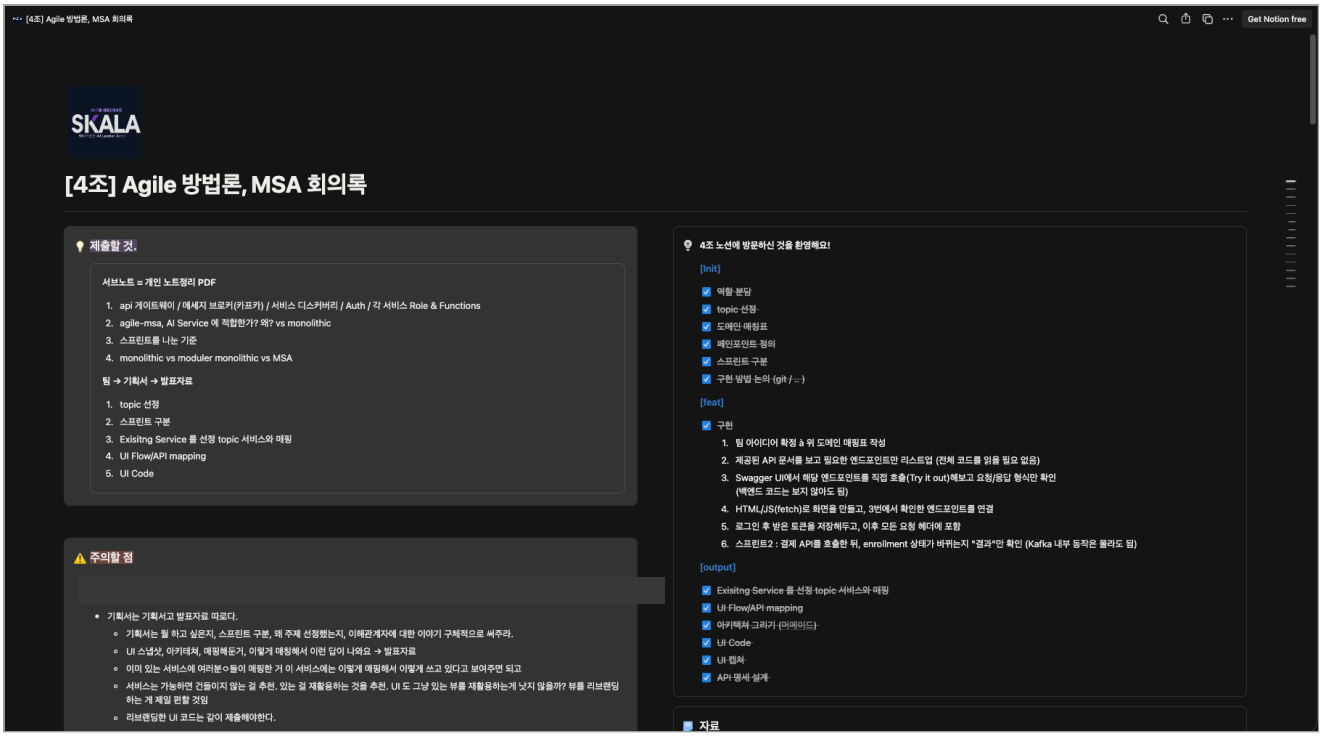
[표 14] Sprint 2 Task 목록

흐름 단계	Task	담당 영역	우선순위
구독 결제	Kafka 토픽 및 이벤트 DTO 치환	Backend	P0 필수
	payment 필드 치환 (courseId → productId)	Backend	P0 필수
	결제 화면의 구독 결제 흐름 연동	Frontend	P0 필수
AI 추천	recommend-service(Python) 치환: 토픽명, 이벤트 키, 호출 경로	Backend	P0 필수
	추천 화면에 사용 패턴 기반 API 추천 노출	Frontend	P0 필수
공통	API 명세 정리 및 Swagger, F12 캡처	기획·문서	P0 필수
	발표 자료: 아키텍처 구성도 (동기 및 비동기 2경로)	기획·문서	P0 필수
	발표 자료: Pain Point, 도메인 매핑표, 스프린트 구분 근거	기획·문서	P0 필수
	Sprint 2 통합 검증: 결제, Kafka, ACTIVE 전환, 추천 갱신	기획·문서	P0 필수
선택 과제	자연어 검색 (임베딩 기반 API 매칭)	Backend	P2 여유되면
	API 성능 비교 화면	Frontend	P2 여유되면

VIII. 협업 운영 방식

8.1 노션 기반 협업

팀의 모든 논의는 노션 한 곳에 모았다. 회의록이자 작업 목록이고 자료 보관소인 단일 창구를 두어, 논의가 메신저 여기저기에 흩어지지 않도록 했다. 의사결정이 필요한 사안은 반드시 노션에 항목을 만든 뒤 이야기를 시작했고, 결정된 내용은 그 자리에 바로 적어 남겼다.



[그림 1] 노션 첫 화면
왼쪽에 제출물 목록과 주의할 점을, 오른쪽에 [Init] / [feat] / [output] 진행 체크리스트를 배치해 무엇을 만들어야 하고 지금 어디까지 왔는지를 한 화면에서 확인할 수 있도록 구성했다.

첫 화면은 목적이 다른 블록 세 개로 나눴다. 왼쪽 위 **제출할 것** 블록에는 개인 서브노트 항목과 팀 제출물(기획서, 발표자료)을 목록으로 적어 두어, 작업 중에도 최종 산출물이 무엇인지 계속 확인할 수 있게 했다. 그 아래 **주의할 점** 블록에는 “AI가 만든 티가 나지 않게 다듬는다”, “B2C 서비스는 지양한다”, “기획서와 발표자료의 역할을 분리한다”, “서비스와 UI는 최대한 재활용한다” 같은 팀 공통 원칙을 적어, 작업 방향이 흔들릴 때 돌아볼 기준으로 삼았다. 오른쪽 **진행 체크리스트** 블록에는 [Init] 단계(역할 분담, topic 선정, 도메인 매칭표, 페인포인트 정의, 스프린트 구분, 구현 방법 논의), [feat] 단계(구현 절차), [output] 단계(서비스 매핑, UI Flow, 아키텍처, UI 코드, API 명세)를 체크박스로 두어, 페이지를 열자마자 팀의 현재 위치가 보이도록 했다. 과제 가이드 PDF와 평가 항목 문서도 같은 페이지에 첨부해 자료를 따로 찾아다닐 필요가 없게 했다.

브레인스토밍 창구

주제 선정 단계에서 노션은 브레인스토밍 공간으로 쓰였다. Topic 아이데이션 항목에 각자 후보 주제를 올리고 제안자 이름을 태그로 달아, 누가 낸 안인지 바로 알 수 있게 했다. 후보마다 **배경, 페르소나, 전체 서비스 시나리오, 서비스 매핑표**를 같은 형식의 토글로 정리하도록 해서, 서로 다른 도메인이라도 같은 기준으로 비교할 수 있었다.



[그림 2] Topic 아이데이션
후보 주제 여섯 건을 제안자 태그와 함께 정리하고, 각 후보를 동일한 항목(배경, 페르소나, 서비스 시나리오, 매핑표)으로 구성해 비교했다. 하단의 투표 블록에서 최종 주제를 확정했다.

올라온 후보는 디지털 물류 플랫폼, AI API 마켓플레이스, 기업용 GPU 클라우드 자원 예약·정산 플랫폼, 기업용 재생에너지 전력 구매 계약(PPA) 플랫폼, 공급망 위험 매칭 플랫폼, 사내 안전교육 플랫폼 여섯 건이었다. 각 후보를 같은 형식으로 정리한 뒤 페이지 하단의 투표 블록에서 팀원별 선택을 적어 **AI API 마켓플레이스**를 최종 주제로 확정했다. 의견을 자유롭게 적는 별도 블록도 두어, 정리된 후보가 아니더라도 떠오르는 생각을 남길 수 있게 했다. 실제로 "주제를 SK에서 고민 중인 것으로 하면 좋지 않을까"라는 의견이 이 블록에서 나와 주제 방향에 반영되었다.

소통 창구로서의 역할

- 결정 사항과 그 근거가 문서에 함께 남아, 나중에 왜 그렇게 정했는지를 다시 확인할 수 있었다.
- 역할 분담(PLAN, FRONT, SERVER)과 담당자별 작업을 같은 페이지에 적어 두어, 누구에게 물어야 하는지가 명확했다.
- 1일차 기획 단계의 논의가 그대로 2일차 개발의 백로그가 되었으므로, 별도의 인수인계 과정이 필요하지 않았다.

8.2 Kanban

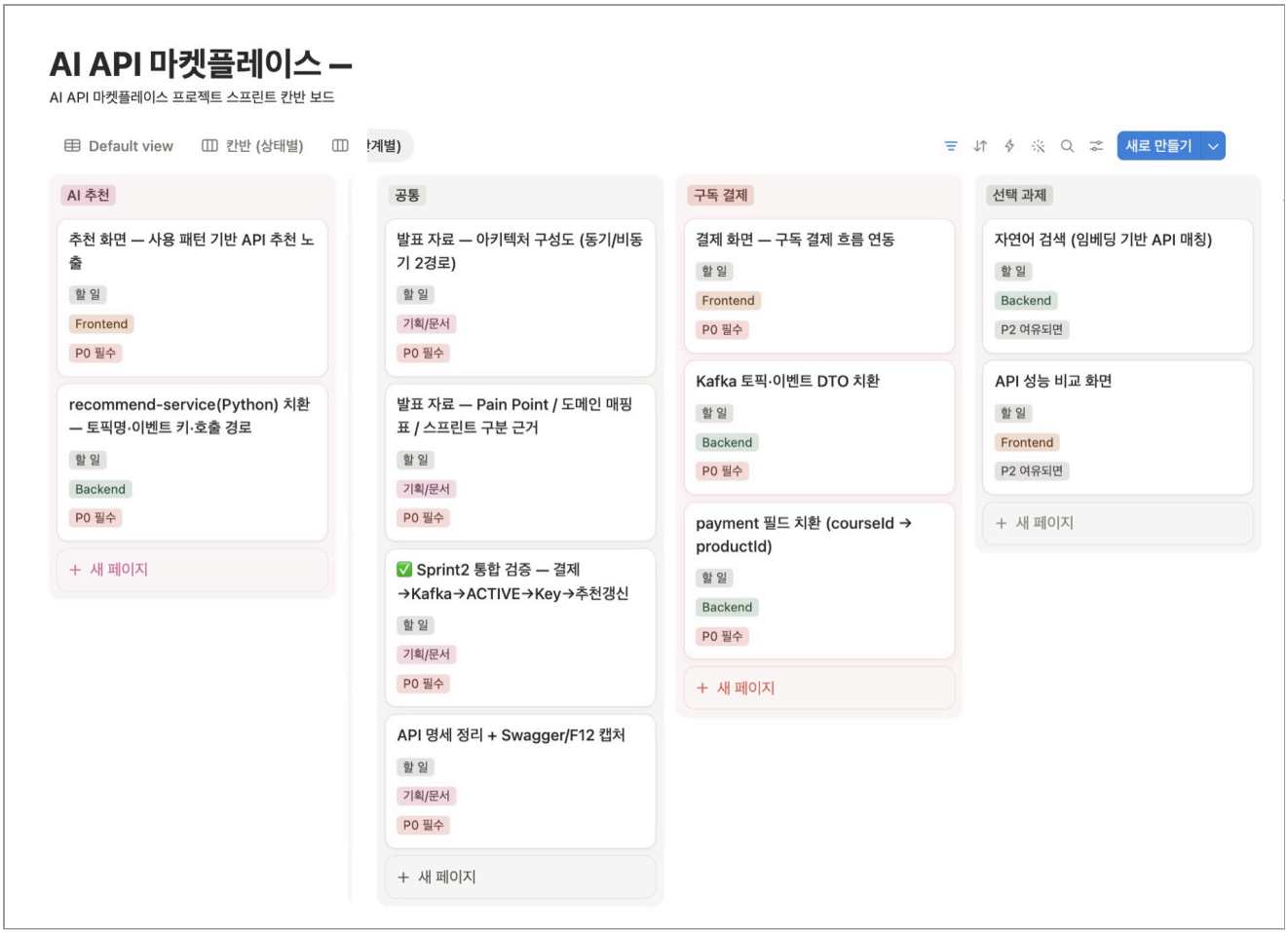
노션 페이지 내 [Init], [feat], [output] 체크리스트와 스프린트별 To-do 리스트로 칸반 보드를 운영했다. 항목마다 체크(완료, 진행중, 미착수) 표시와 "완료", "미사용", "미구현" 같은 상태 태그를 달아, 팀 전체가 지금 무엇을 하고 있는지 한눈에 파악할 수 있도록 관리했다.

보드는 상태별 뷰와 흐름 단계별 뷰를 함께 두었다. 상태별 뷰로는 진행률을 확인하고, 흐름 단계별 뷰(회원가입, API 등록, 카탈로그 조회, 구독 신청, 공통)로는 한 기능이 백엔드부터 프론트까지 어디까지 왔는지를 확인했다.



[그림 3] Sprint 1 칸반 보드 (흐름 단계별 뷰)

회원가입, API 등록, 카탈로그 조회, 구독 신청, 공통 다섯 단계로 나누고 카드마다 담당 영역과 우선순위 태그를 달았다.



[그림 4] Sprint 2 칸반 보드 (흐름 단계별 뷰)
AI 추천, 공통, 구독 결제, 선택 과제로 구성했고 선택 과제는 P2로 표시해 여유가 있을 때만 착수하도록 구분했다.

8.3 Daily Scrum

프로젝트는 이틀에 걸쳐 진행했고, 1일차는 기획, 2일차는 개발과 발표 준비에 배정했다. 스크럼은 정해진 횟수를 채우기보다 작업 단계가 넘어갈 때마다 수시로 모이는 방식으로 운영했고, 매번 지금 누가 무엇을 하고 있는지를 확인했다.

1일차 — 기획

노션에 서비스 기획 문서를 함께 작성하며 각자 의견을 내어 주제를 도출했다. 이어서 해당 서비스의 페인포인트를 정리하고, 그 페인포인트를 AI로 어떻게 해결할 수 있는지 논의해 솔루션 방향을 확정했다. 마지막으로 확정된 솔루션에서 구현할 feature를 뽑아 백로그로 정리하며 1일차를 마쳤다.

2일차 — 개발 및 발표 준비

개발은 백엔드에서 프론트로 이어지는 순서로 시간을 나눠 진행했다. 오전 10시까지 백엔드 작업을 마무리해 프론트가 붙을 수 있는 API를 확보하고, 오후 1시까지 프론트에서 Sprint 1 범위를, 오후 3시까지 Sprint 2 범위를 완료하는 일정으로 잡았다. 남은 3시부터 4시까지는 발표 준비와 동작 화면 스크린샷 수집에 사용했다.

[표 15] 일자별 진행 계획 및 스크럼 확인 사항

일차	시간	진행 내용	스크럼 확인 사항
1일차	전일	노션에 서비스 기획 문서 작성, 의견 교환을 통한 주제 도출, 페인포인트 정리, AI 솔루션 방향 확정, feature 도출	주제와 범위에 팀 전원의 의견이 반영되었는지 확인
2일차	~ 10:00	백엔드 도메인 치환 및 API 명세 확정	프론트가 붙을 수 있는 API가 확보되었는지 확인
2일차	~ 13:00	프론트에서 Sprint 1 범위 구현 (회원가입부터 구독 신청까지)	핵심 흐름이 한 바퀴 통과되는지 확인
2일차	~ 15:00	Sprint 2 범위 구현 (결제, 카테고리 필터, 추천)	미완 항목을 확인하고 이월 여부 결정

일차	시간	진행 내용	스크럼 확인 사항
2일차	15:00 ~ 16:00	발표 준비, 동작 화면 스크린샷 수집	제출물 누락 여부 확인

작업의 크기와 진행도는 Story Point로 관리했다. 스크럼마다 완료된 스토리의 SP를 더해 남은 SP와 대조하면, 지금 어느 정도 분량이 남았고 계획 대비 얼마나 진행됐는지가 한눈에 보였다. 그 결과가 **[표 12] 스프린트별 진행 현황**이다. Sprint 1은 계획 24점을 전량 소진(달성률 100%)했고, Sprint 2는 18점 중 13점을 소진(72%)한 뒤 남은 5점(US-09)을 다음 스프린트로 이월하기로 스크럼에서 결정했다.

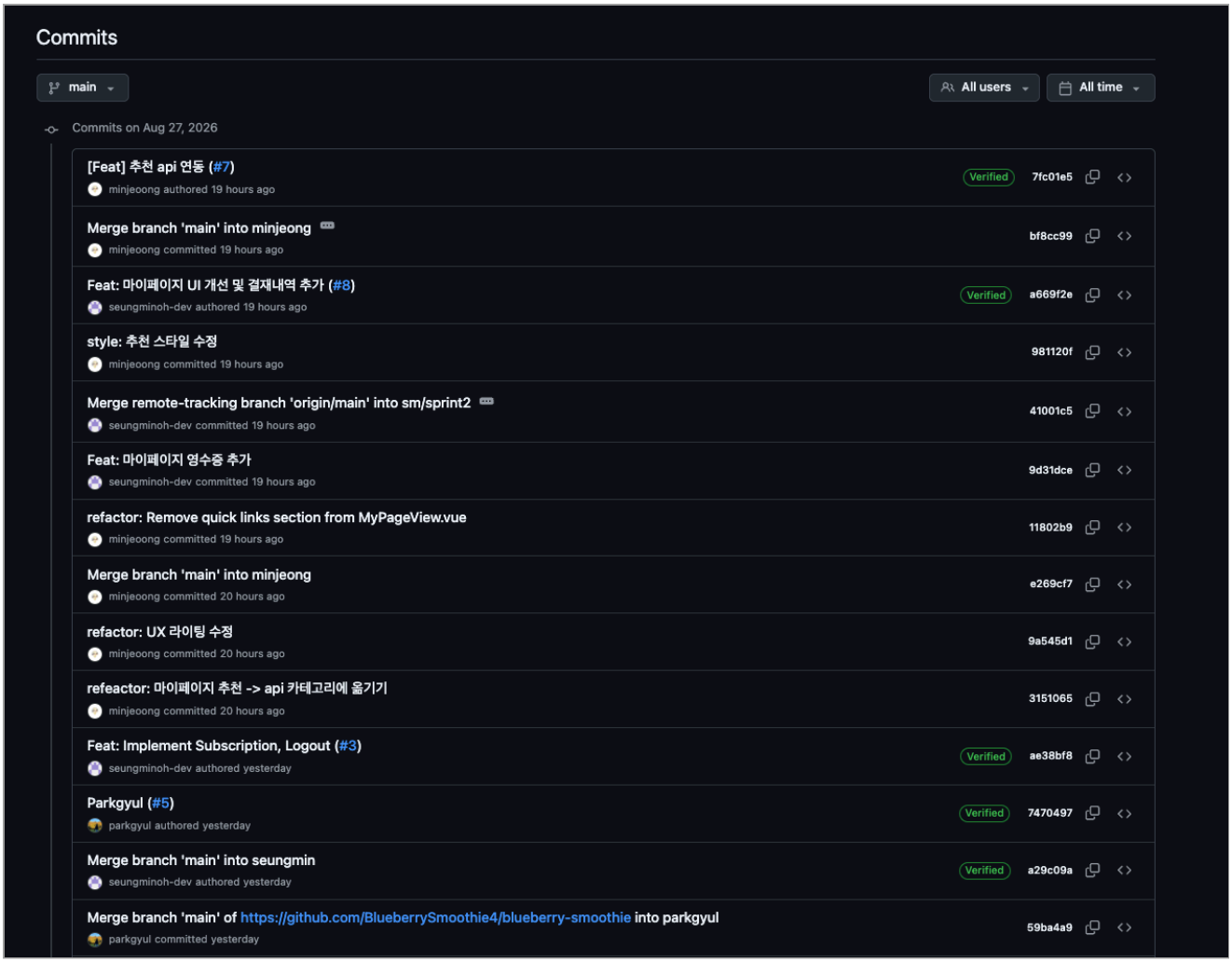
8.4 형상 관리

개발은 전 과정을 GitHub 저장소(BlueberrySmoothie4/blueberry-smoothie)에서 진행했다. 팀원마다 개인 브랜치(`minjeong`, `seungmin`, `parkgyul`, `sm/sprint2`)를 두고 각자 작업한 뒤, `main`으로 Pull Request를 올리고 다른 팀원이 변경 내용을 확인한 다음 머지하는 방식으로 통합했다. 2일이라는 짧은 기간이었지만 `main` 기준 39개 커밋, 6건의 PR, 17회의 머지가 쌓였다.

개별 커밋 메시지에는 `[Feat]`, `feat:`, `fix:`, `refactor:`, `style:` 같은 접두어를 붙여 변경의 성격을 한눈에 구분할 수 있게 했다. 커밋 그래프와 PR 목록 자체가 진행 현황판 역할을 해서, 스크럼 시간이 아니어도 누가 어떤 작업을 어디까지 진행했는지 파악할 수 있었다. 충돌이 발생한 경우에는 커밋 메시지에 `[Conflict]`를 남겨 해결 이력을 추적할 수 있게 했다.

[표 16] main 브랜치 병합 PR 목록

PR	제목	작성자	주요 내용
#1	[Feat] 디자인 수정	김민정	홈 화면 스타일 적용, 브랜드 색상 초기 반영
#3	Feat: Implement Subscription, Logout	오승민	구독 신청, 내 구독 조회, 로그아웃 기능 구현
#4	[Feat] 상세 페이지 및 디테일 드로어 추가	김민정	상품 상세 화면, 상세 드로어 전환, 카테고리 글리프
#5	Parkgyul	박규리	초기 SQL 작성, 도메인 변경 반영
#7	[Feat] 추천 API 연동	김민정	recommend-service 연동, 카탈로그 상단 추천 영역
#8	Feat: 마이페이지 UI 개선 및 결제내역 추가	오승민	마이페이지 개선, 결제 내역 및 영수증 화면



[그림 5] GitHub 커밋 목록 (main 브랜치, 2026-08-27)
2일차 개발 당일의 커밋 이력. 제목에 (#7)처럼 PR 번호가 붙은 것이 Pull Request를 통해 병합된 커밋이고, 그 사이의 항목은 각 팀원이 개인 브랜치에서 작업한 내역이다. 하루 동안의 작업 흐름과 팀원별 기여가 시간순으로 드러난다.

8.5 트러블슈팅 기록

개발 중 발생한 이슈는 증상, 원인, 조치, 재발 방지 항목으로 정리했다. 원인을 적지 못하면 해결한 것으로 보지 않기로 했다. 두 스프린트를 통틀어 배포 단계에서 가장 큰 이슈가 있었다.

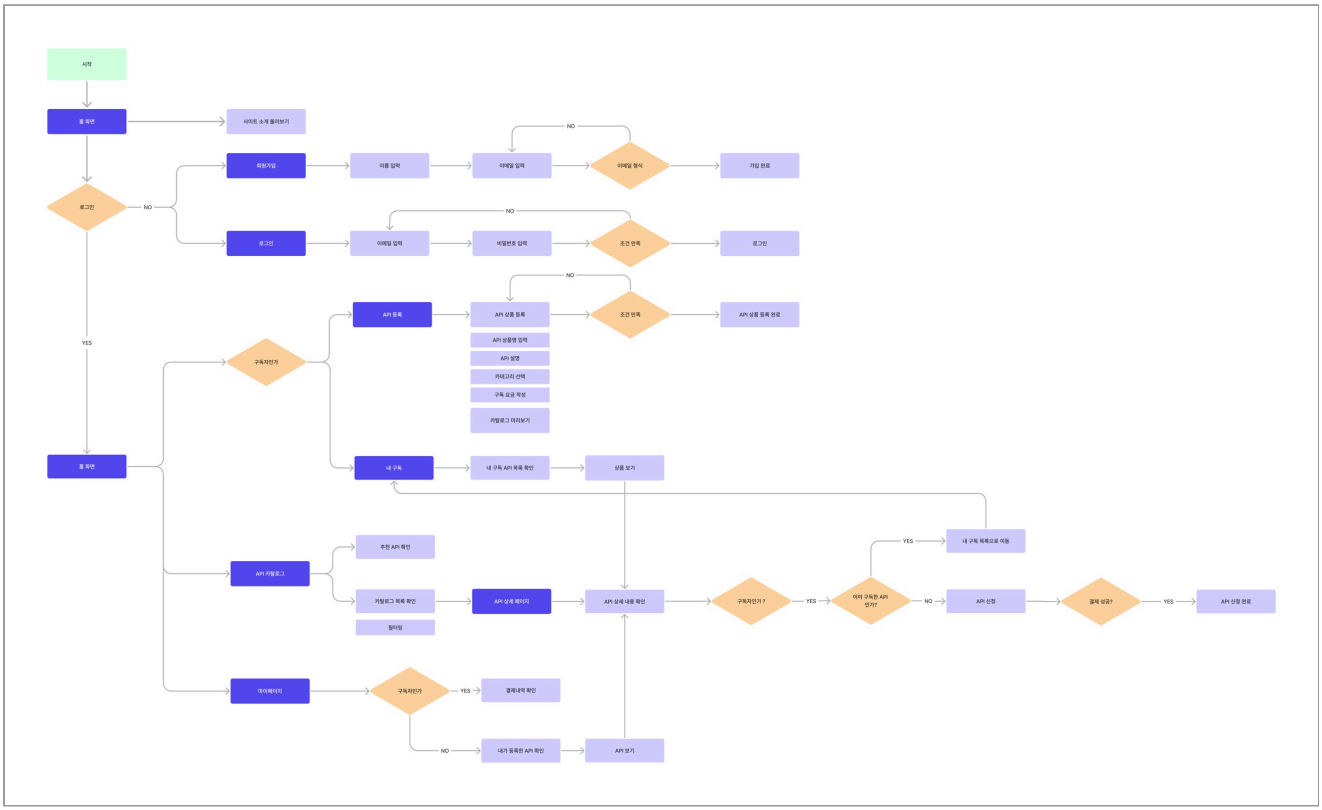
[표 17] 트러블슈팅 기록 — 도커 이미지 배포 반영 실패

항목	내용
증상	코드를 수정한 뒤 도커 이미지를 다시 빌드해 배포해도, 실행 중인 컨테이너가 최신 버전으로 갱신되지 않았다. 도커를 내리고 재빌드한 다음 다시 올리는 과정을 여러 번 반복했지만 변경 사항이 반영되지 않았다.
원인	인프라 이미지를 레지스트리에서 직접 내려받는 과정에서 접속이 몰려 `429 Too Many Requests`가 반복적으로 발생했다. 이를 피하기 위해 강사가 안내한 우회 방식(필요한 이미지를 하나의 `tar` 파일로 묶어 로컬에서 로드하는 방식)으로 배포하고 있었는데, 이 방식이 로컬에 적재된 `tar` 이미지를 그대로 사용해 새로 빌드한 이미지를 반영하지 않았다.
조치	도커 down · 재빌드 · up을 반복해도 해결되지 않아, 배포 방식 자체를 되돌리기로 했다. 우회책 대신 레지스트리에서 이미지를 정상적으로 내려받는 정석 방식으로 바꾸되, 429를 피하기 위해 휴대폰 핫스팟으로 네트워크(IP)를 변경한 뒤 이미지를 받았다. 이 방식으로 배포하니 최신 코드가 정상 반영되었다.
해결 주체	화면 증상만으로는 코드 문제인지 배포 문제인지 구분되지 않아, 백엔드와 프론트엔드 팀원이 함께 모여 원인을 좁히고 해결했다.
재발 방지	배포 우회책을 도입할 때는 그 방식이 이후 재배포에 어떤 부작용을 남기는지 먼저 확인한다. 네트워크 제약(429)은 우회 스크립트를 짜기보다 회선을 바꾸는 편이 부작용이 적다.

IX. 화면 설계

Figma로 주요 화면을 설계하고 그 기준으로 UI를 구현했다. 아래는 실행 중인 서비스에서 직접 캡처한 화면이다. SK AM의 계정은 회원가입 시 역할을 **구매 기업 담당자(BUYER)** 또는 **API 제공사(PROVIDER)** 중 하나로 선택하며, 로그인 이후에는 이 역할에 따라 사이드바 메뉴, 노출 화면, 사용할 수 있는 기능이 갈린다. 권한은 화면에서만 숨기는 것이 아니라 라우터 가드로도 강제된다 (미인증 시 로그인으로, 로그인 상태로 로그인 화면 접근 시 카탈로그로, 제공사 전용 화면에 구매 기업이 접근하면 카탈로그로 되돌린다).

전체 사용자 흐름은 [그림 6]에, 개별 화면과 대응 User Story는 [표 18]에 정리했다. 흐름 상 **로그인** 분기에서 비로그인 사용자는 회원가입·로그인으로, 로그인 사용자는 역할(**구독자인가?**)에 따라 구독자는 카탈로그·상세·구독·마이페이지로, 제공사는 API 등록·내가 등록한 API로 갈라진다.



[그림 6] 전체 사용자 흐름

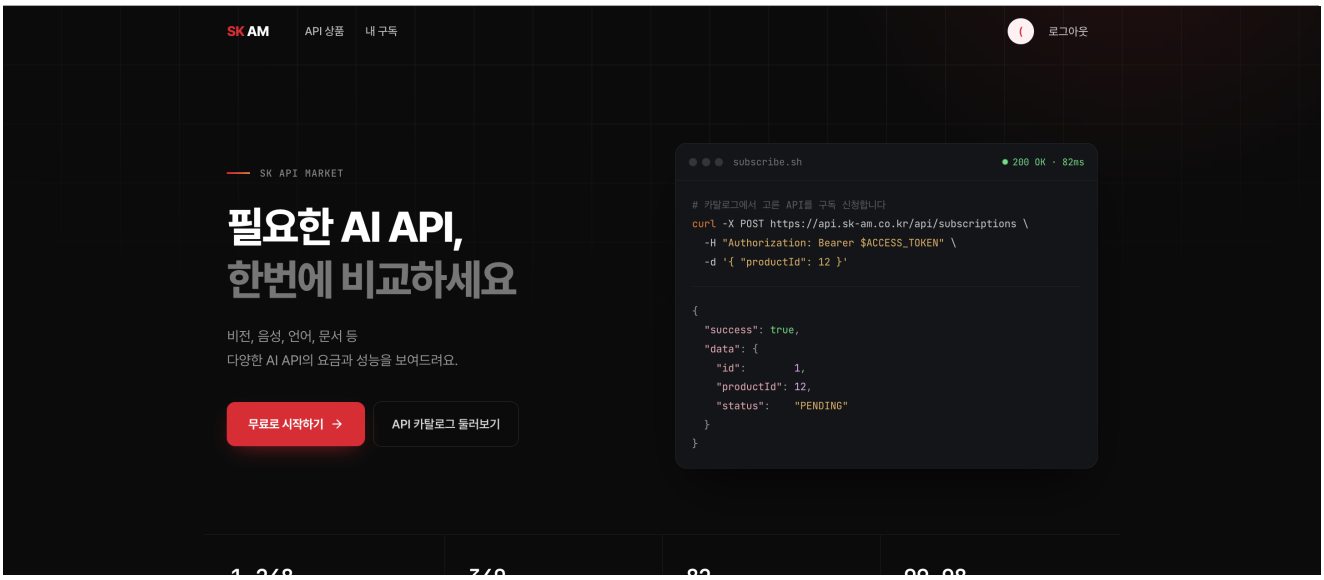
홈 → 로그인 분기 → (구독자) 카탈로그·상세·구독 신청·결제 성공·내 구독 / (제공사) API 상품 등록·내가 등록한 API. 결제 성공 시 내 구독 목록으로 이동한다.

[표 18] 화면 목록 및 접근 계정

그림	화면	접근 계정	대응 US	핵심 내용
7·8	메인(랜딩)	비로그인	—	서비스 소개와 지표, 구매 기업·제공사 두 갈래 진입점
9·10	로그인 · 회원가입	비로그인	US-01·02·03	OAuth2 로그인, 역할 선택 회원가입, 입력 검증
11·12	API 목록(카탈로그)	공통 (역할별 차이)	US-05·08	카테고리 필터·상품 카드, 역할에 따라 추천/등록 버튼 분기
13	API 상세 · 구독 신청	BUYER	US-05·06	스펙 표, 요금 패널, 구독 신청 요청 예시, 상태별 버튼
14	내 구독	BUYER	US-07	구독 중인 API 목록, 상태 배지, 구독 번호
15	마이페이지 · 결제 내역	BUYER	US-10·11	월별 결제 내역, 누적 결제·구독 수 요약
16	API 상품 등록	PROVIDER	US-04	등록 폼, 카탈로그 미리보기, 입력 검증
17	제공사 대시보드	PROVIDER	US-12	등록한 API별 구독 기업 수, 총 구독 기업 수

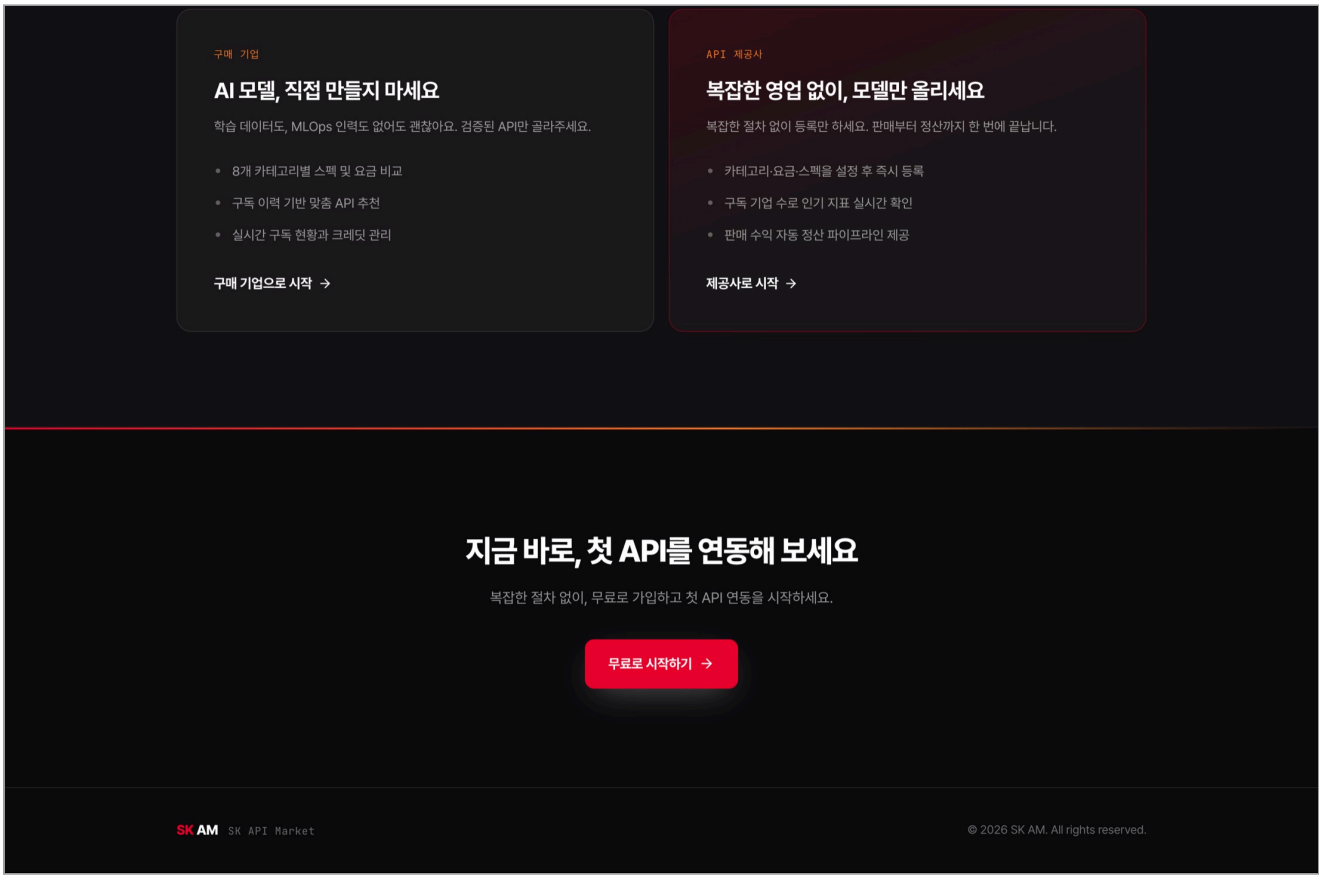
9.1 메인 화면

로그인하지 않은 상태에서 보이는 랜딩 화면이다. 상단은 서비스 소개와 등록 API 수·제공사 수·평균 응답 속도·안정성 지표를 보여주고, 하단에서 **구매 기업으로 시작**과 **제공사로 시작** 두 개의 진입점으로 계정 유형이 갈린다. 각 진입점 아래에 그 역할이 얻는 가치(구매 기업: 스펙·요금 비교, 구독 이력 기반 추천, 크레딧 관리 / 제공사: 즉시 등록, 구독 기업 수 지표, 자동 정산)를 함께 노출한다.



[그림 7] 메인 화면 — 상단

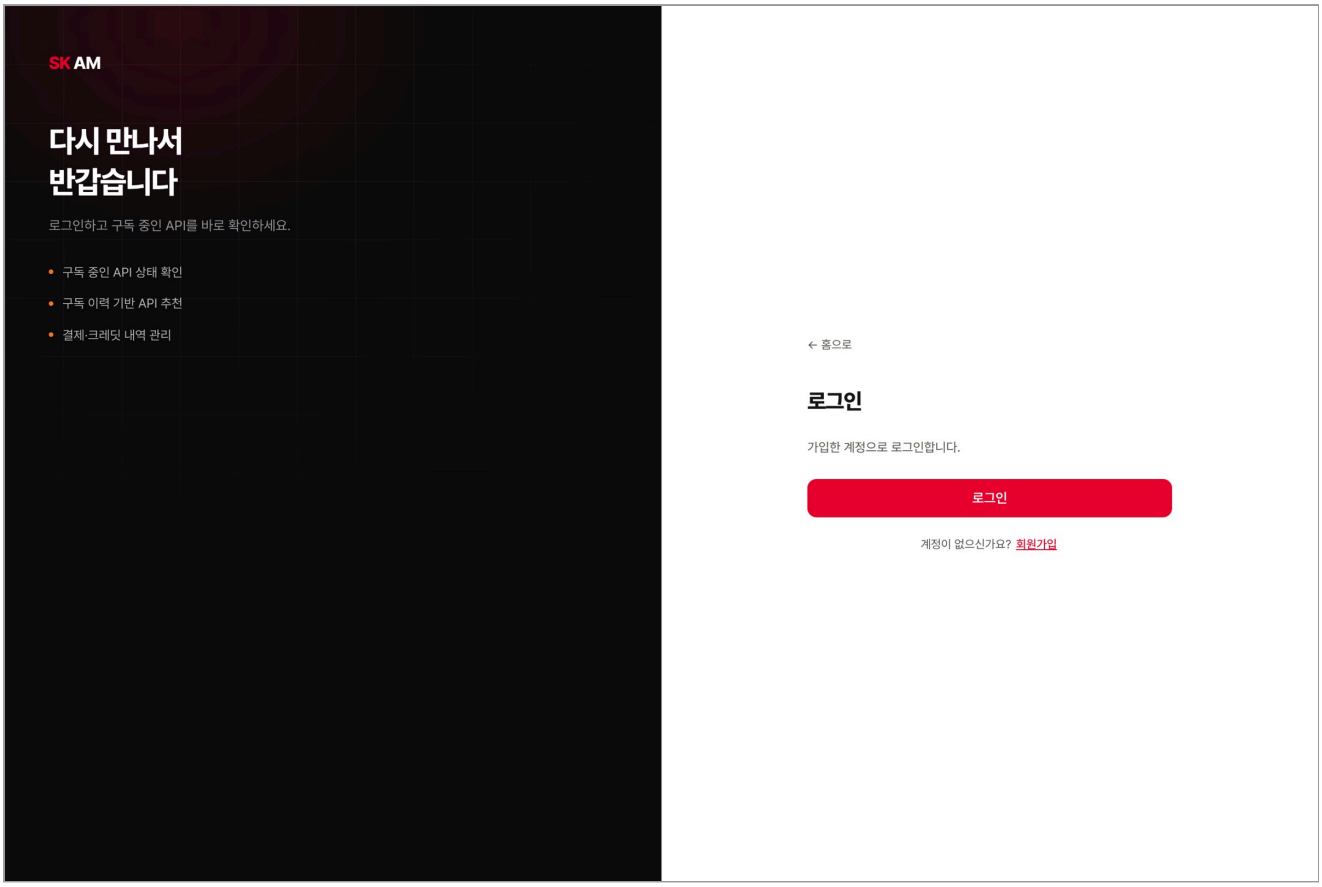
서비스 소개, 실시간 구독 요청 예시, 등록 API·제공사·응답 속도·안정성 지표, "지금 많이 쓰는 API" 목록.



[그림 8] 메인 화면 — 하단 (계정 유형 진입점)

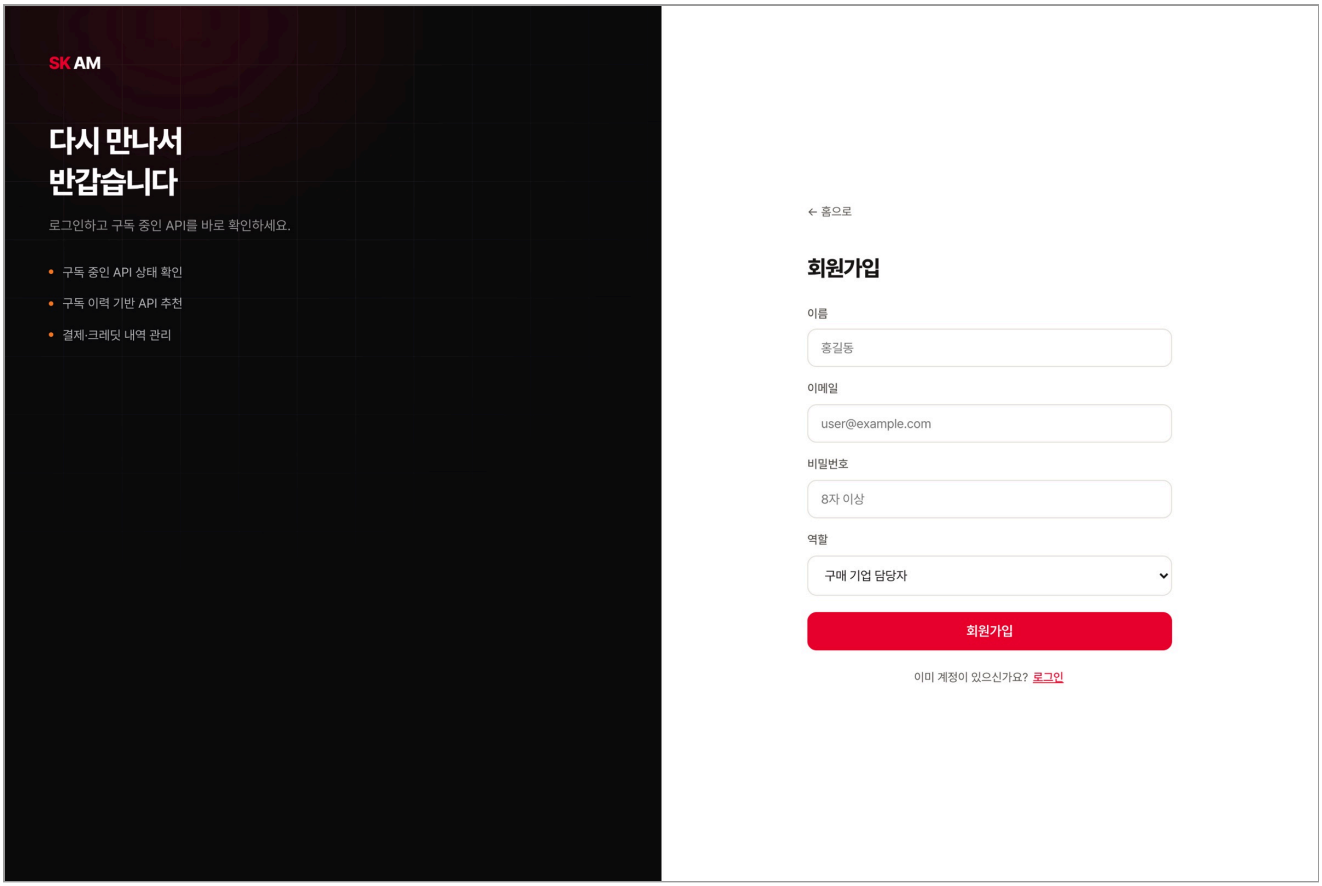
왼쪽은 구매 기업, 오른쪽은 API 제공사 진입 카드. 이 화면에서부터 두 계정의 여정이 갈린다.

9.2 로그인 · 회원가입



[그림 9] 로그인 화면

"로그인" 버튼을 누르면 인증 서버로 이동해 자격 증명을 입력하고, 콜백에서 토큰을 교환한 뒤 내 정보를 불러온다(OAuth2 Authorization Code Flow).



[그림 10] 회원가입 화면

이름·이메일·비밀번호와 함께 **역할**(구매 기업 담당자 / API 제공사)을 선택한다. 선택한 역할이 이후 모든 화면 구성을 결정한다.

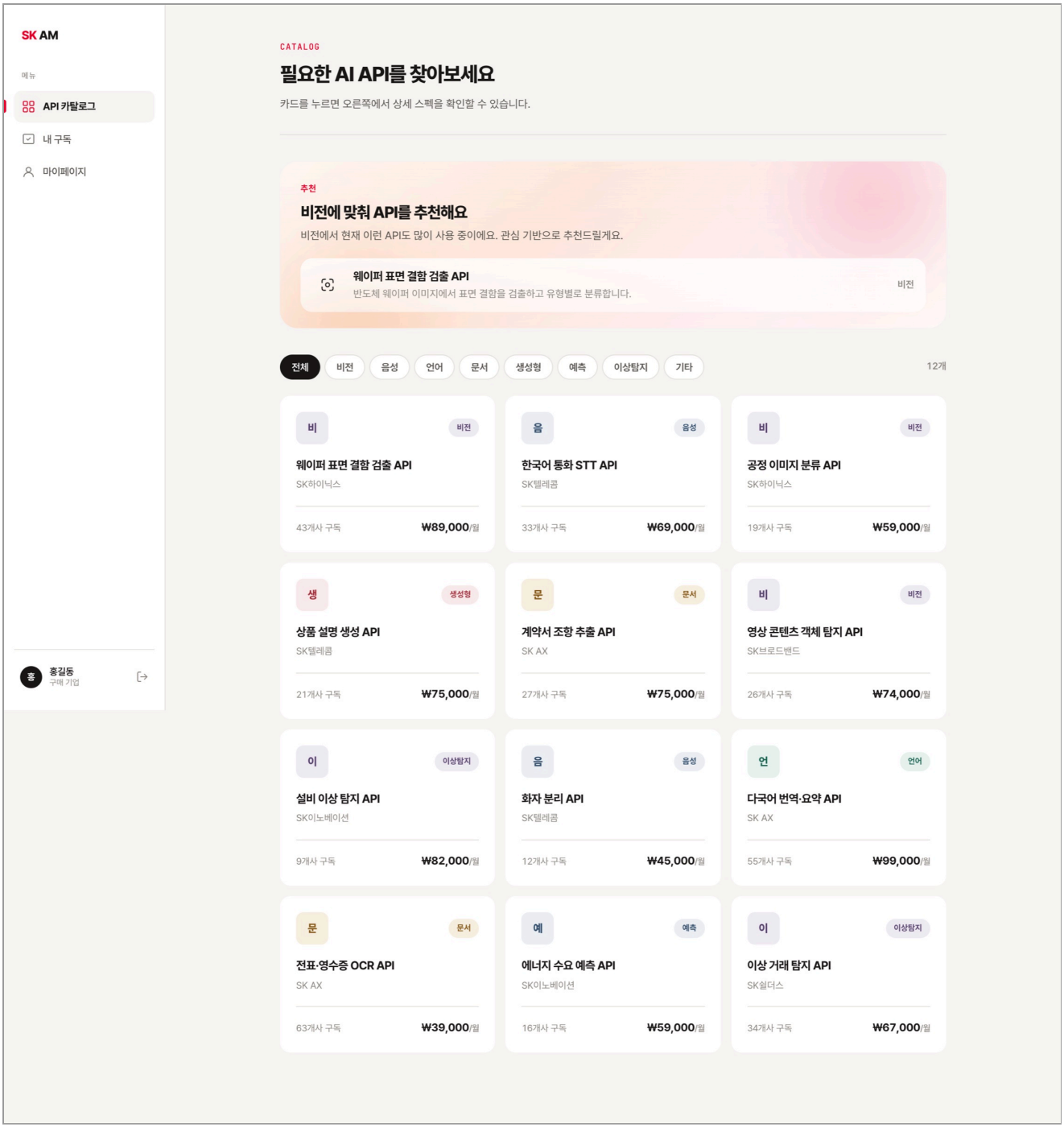
[표 19] 회원가입 입력 검증 정책 (user-service)

항목	규칙	실패 메시지
이메일	필수, 이메일 형식(`@Email`)	"올바른 이메일 형식이 아닙니다"
비밀번호	필수, 8자 이상(`@Size(min=8)`), 입력창 안내 "8자 이상"	"비밀번호는 8자 이상이어야 합니다"
이름	필수	"이름은 필수입니다"
역할	구매 기업 담당자 / API 제공사 중 선택 (미지정 시 구매 기업 담당자)	—
이메일 중복	이미 가입된 이메일이면 가입 거부	"이미 사용 중인 이메일입니다"

> 가입 성공 시 "회원가입 완료! 로그인 페이지로 이동합니다."를 띄우고 2초 뒤 로그인 화면으로 전환한다.

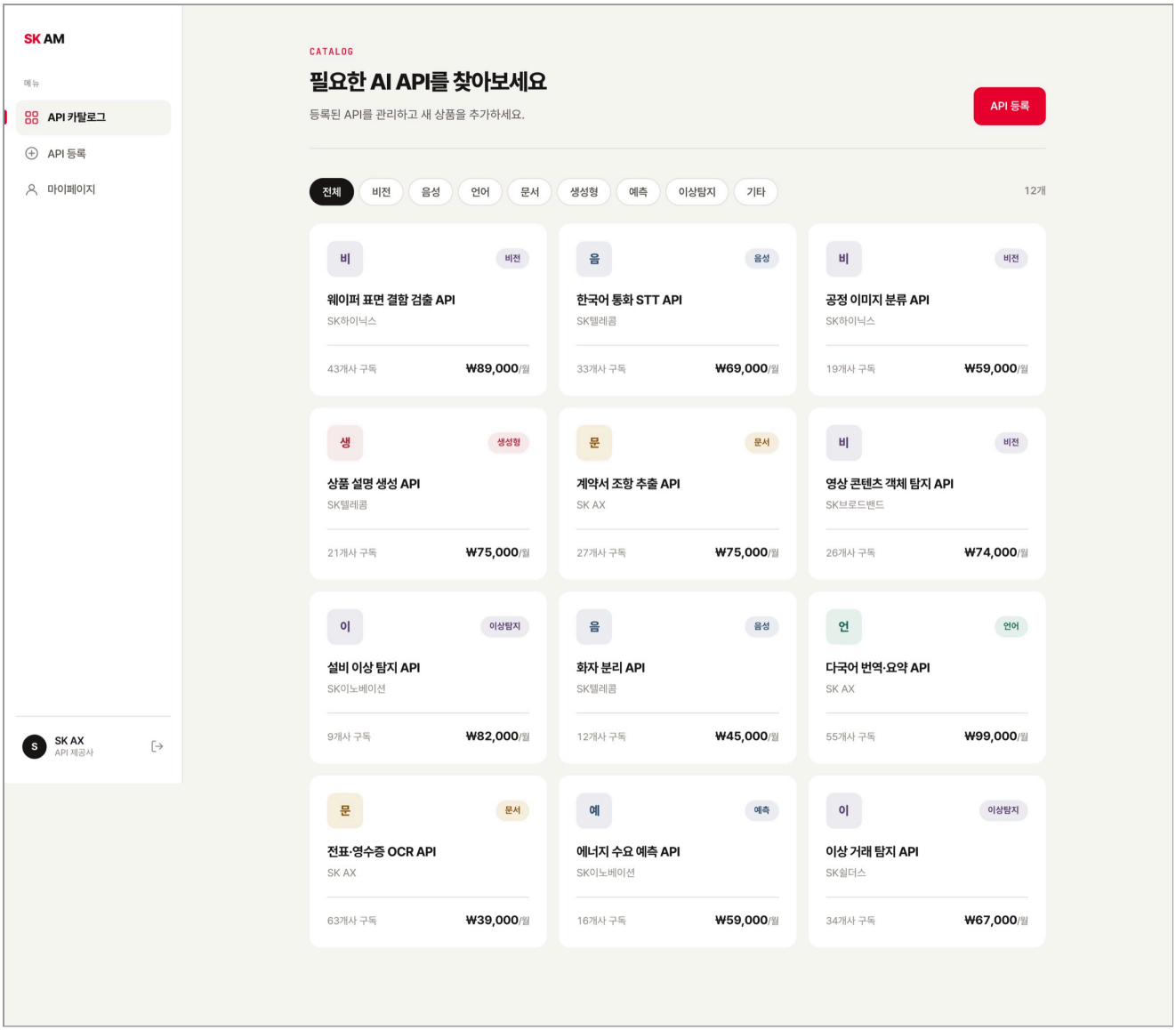
9.3 API 목록 (카탈로그)

같은 `/products` 경로지만 계정 역할에 따라 다르게 보인다. 공통 요소는 카테고리 필터(전체 + 8종)와 결과 개수, 카드 클릭 시 상세 이동이다. 차이는 다음과 같다.



[그림 11] API 카탈로그 — 구매 기업(BUYER) 계정

상단에 "비전에 맞춰 API를 추천해요" 추천 영역이 있고, 사이드바 메뉴는 API 카탈로그 · 내 구독 · 마이페이지이다.



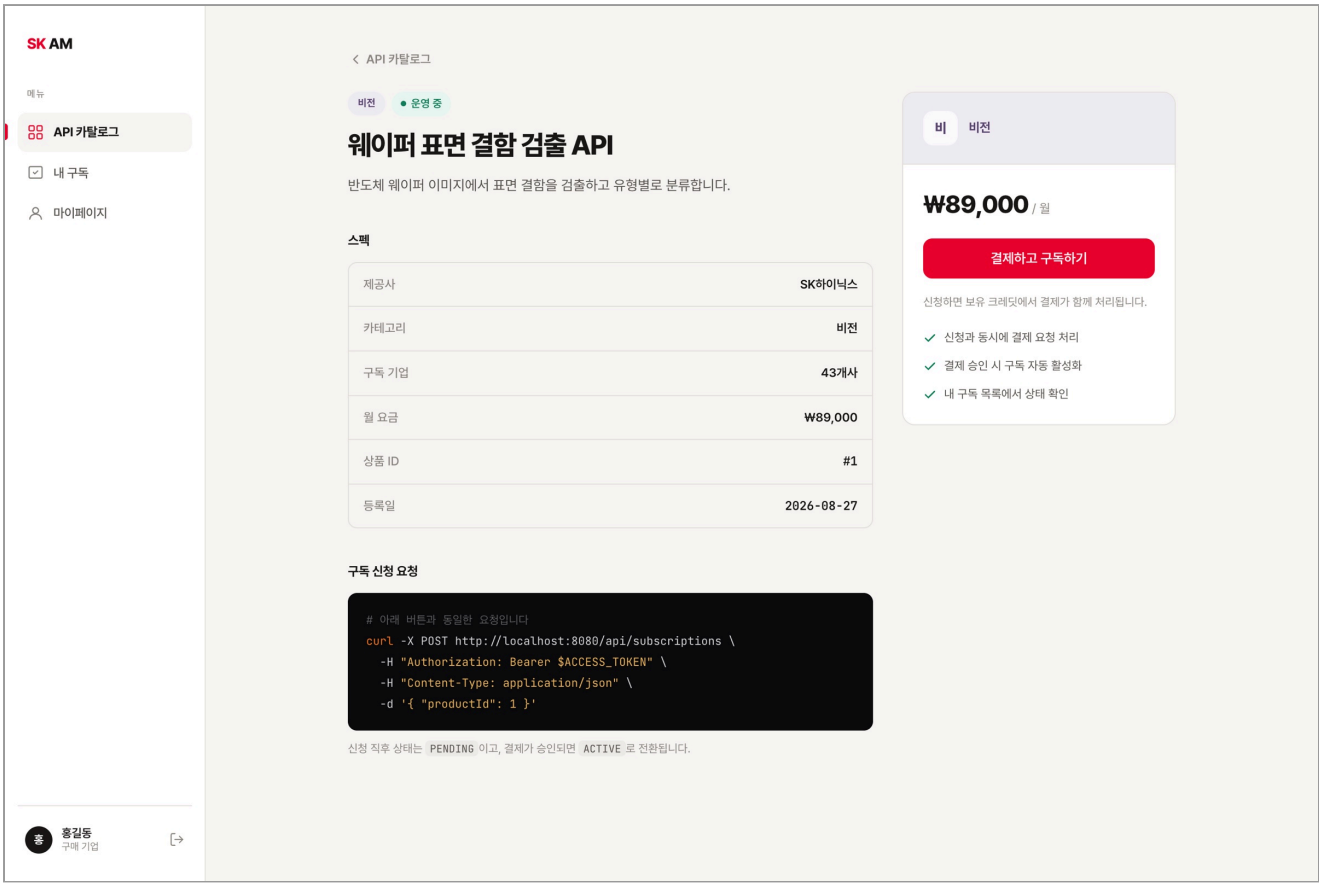
[그림 12] API 카탈로그 — API 제공사(PROVIDER) 계정

추천 영역이 없고, 우측 상단과 사이드바에 "API 등록" 버튼이 있다. 안내문도 "등록된 API를 관리하고 새 상품을 추가하세요."로 바뀐다.

[표 20] 계정 유형별 화면 차이

영역	구매 기업 담당자 (BUYER)	API 제공사 (PROVIDER)
사이드바 메뉴	API 카탈로그 · 내 구독 · 마이페이지	API 카탈로그 · API 등록 · 마이페이지
카탈로그 추천 영역	노출 (사용 이력 기반 추천)	없음
카탈로그 우측 버튼	없음	“API 등록”
상세 화면 CTA	“결제하고 구독하기”	“제공사 계정은 구독 불가” (비활성)
마이페이지	결제 내역 (영수증 목록)	대시보드 (내가 등록한 API)
전용 경로	`/subscriptions`	`/products/new` (라우터 가드로 강제)

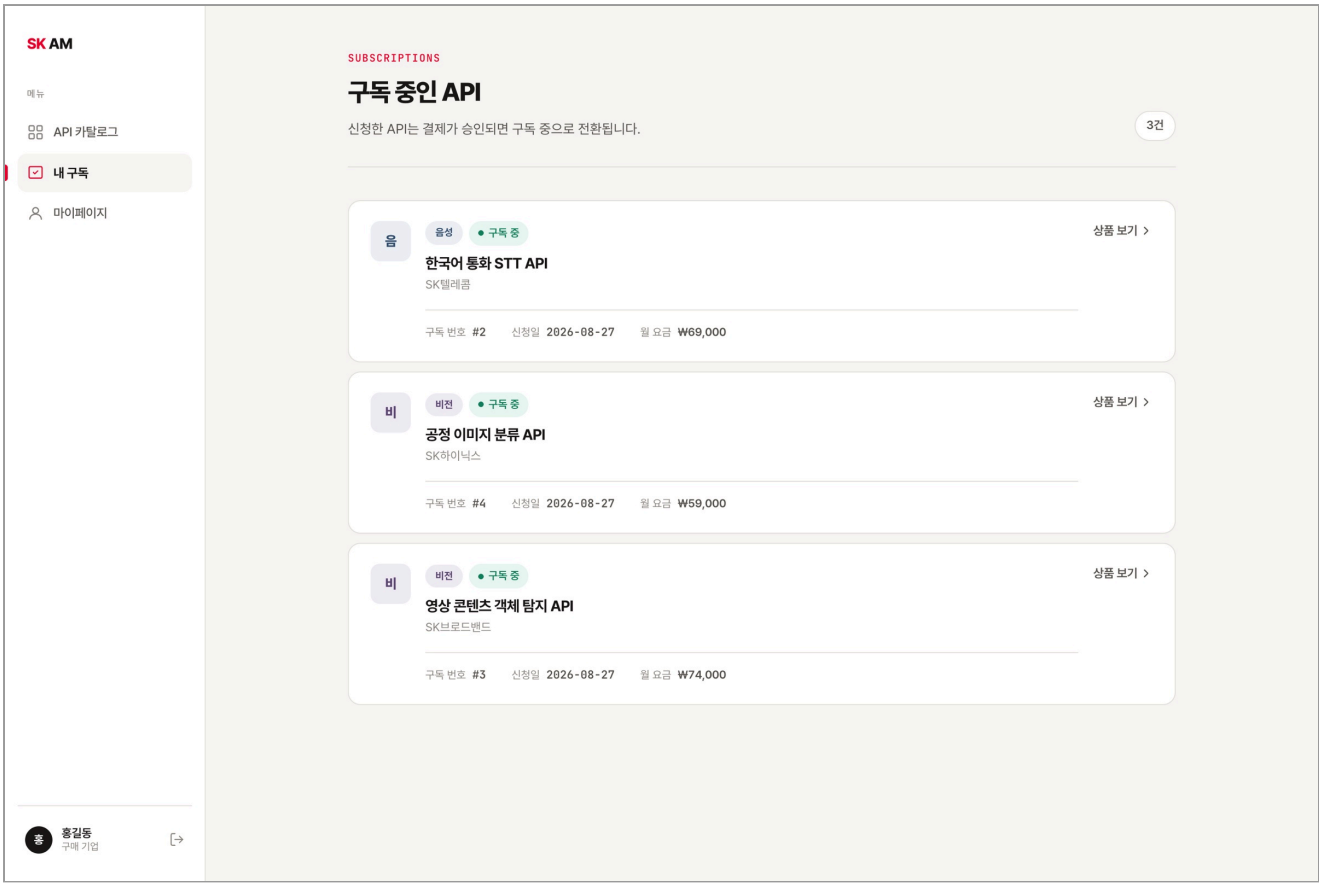
9.4 API 구독자(BUYER) 관점



[그림 13] API 상세 · 구독 신청
왼쪽에 스펙 표(제공사·카테고리·구독 기업 수·월 요금·상품 ID·등록일), 오른쪽에 요금 패널과 "결제하고 구독하기" 버튼, 아래에 실제 구독 신청 요청(curl) 예시를 둔다.

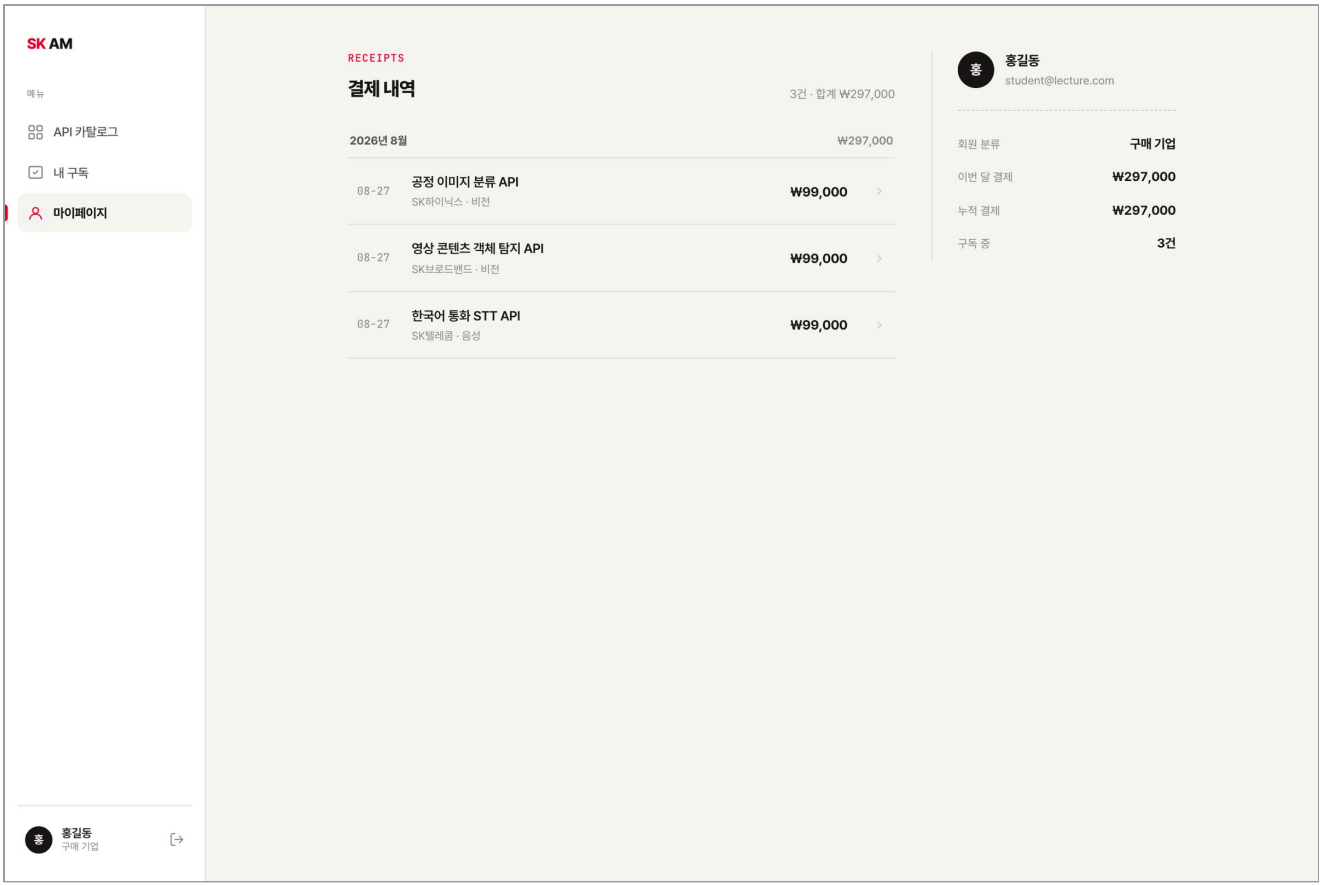
[표 21] 구독 상태 전이 및 버튼 정책

상황	버튼 표시	동작 / 안내
미구독 (BUYER)	“결제하고 구독하기” (활성)	신청과 동시에 결제 요청 → 상태 PENDING 생성
신청 직후	“신청 완료 · 결제 처리 중” (비활성)	상태 PENDING, 결제 승인 대기
결제 승인 후	“내 구독 목록으로 이동”	상태 ACTIVE, “이미 구독 중인 API입니다”
제공사(PROVIDER) 계정	“제공사 계정은 구독 불가” (비활성)	“제공사 계정으로는 구독을 신청할 수 없습니다”



[그림 14] 내 구독

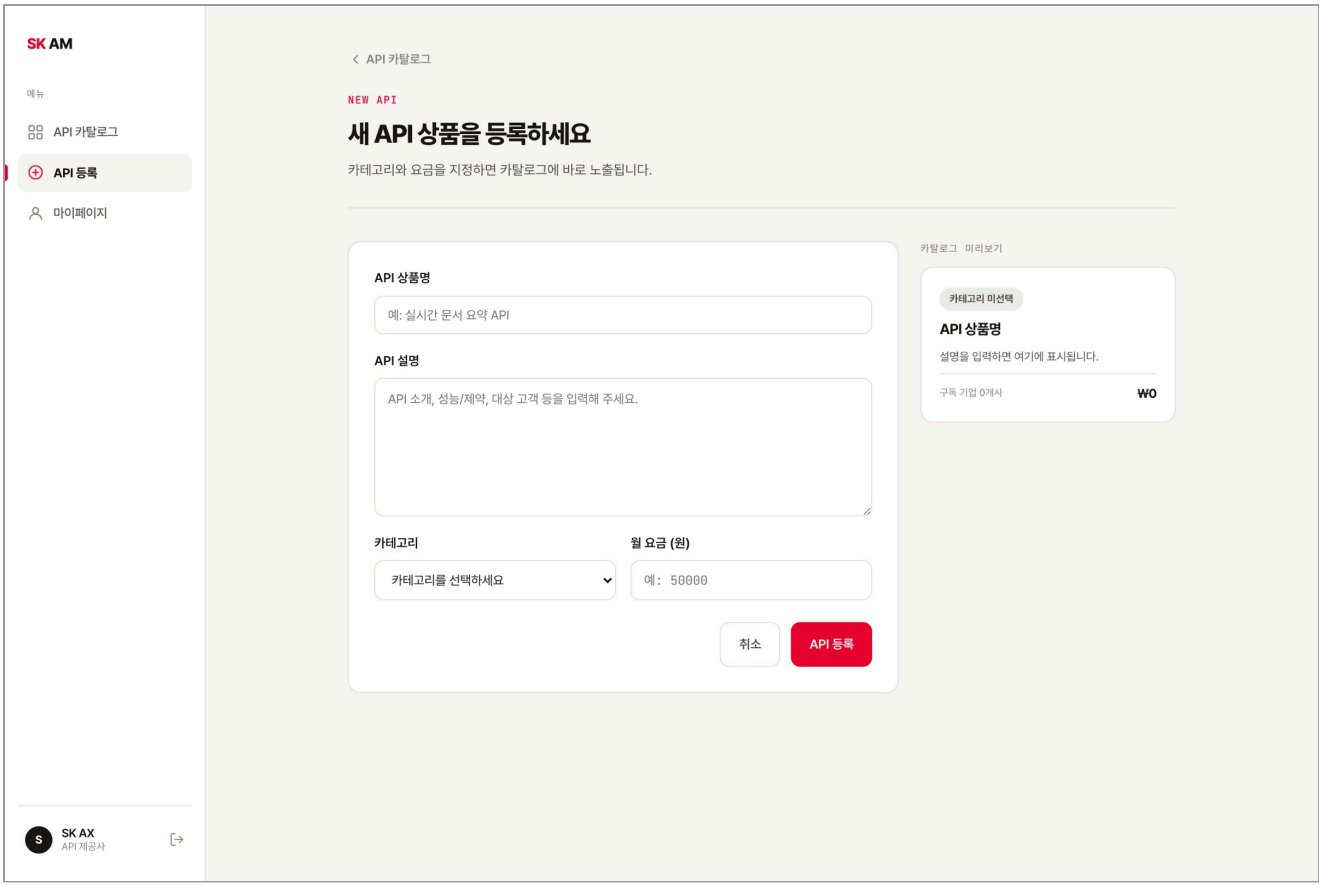
구독 중인 API를 카드로 나열한다. 상태 배지(결제 처리 중 / 구독 중 / 해지됨), 구독 번호, 신청일, 월 요금을 표시한다. 제공사 계정에는 이 메뉴가 없다.



[그림 15] 마이페이지 · 결제 내역

월별 영수증 목록과, 오른쪽에 회원 분류(구매 기업)·이번 달 결제·누적 결제·구독 수 요약. 구매 기업 계정 전용 화면이다.

9.5 API 제공사(PROVIDER) 관점

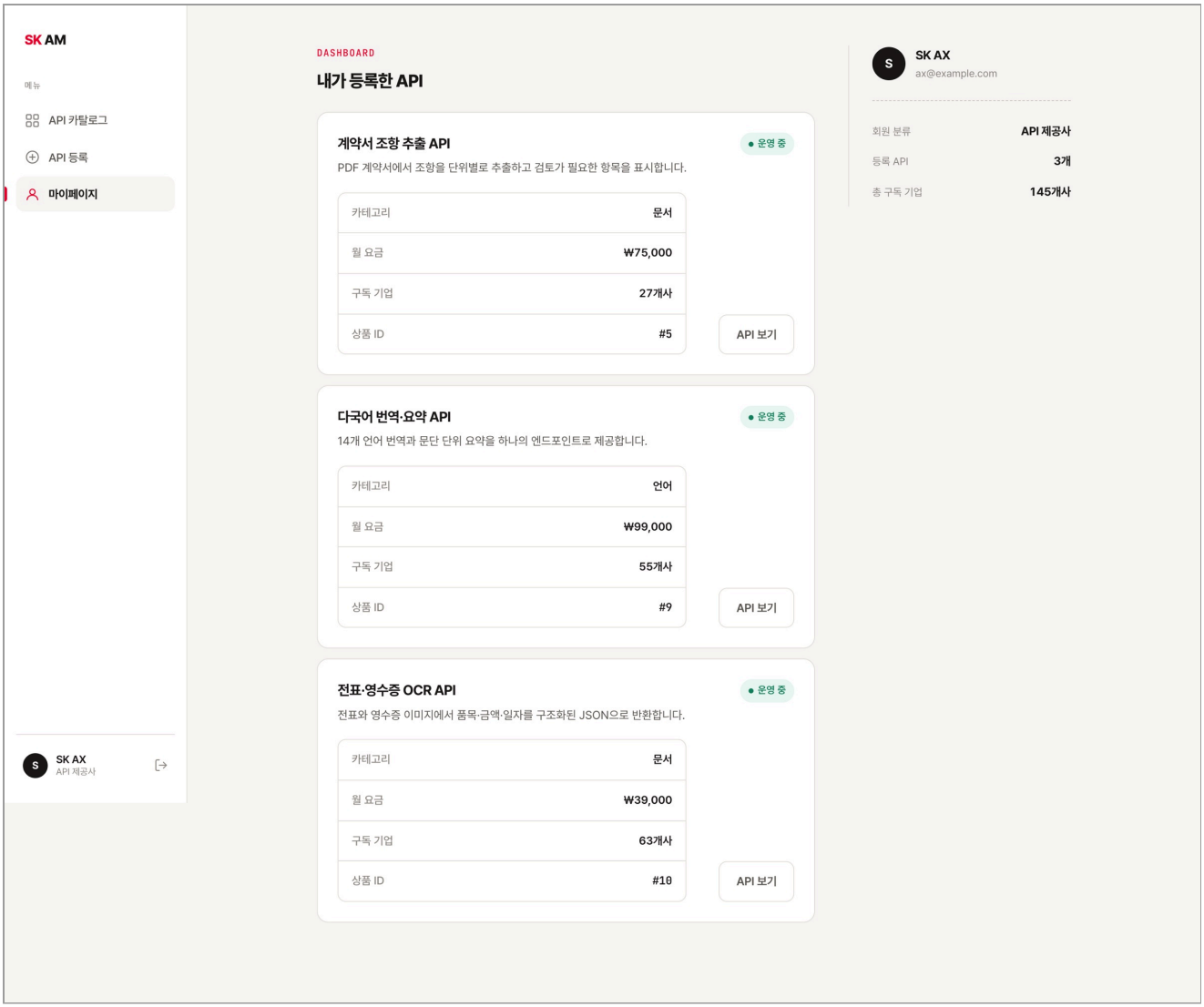


[그림 16] API 상품 등록

상품명·설명·카테고리·월 요금을 입력하면 오른쪽 "카탈로그 미리보기"에 실시간 반영된다. 등록하면 즉시 `PUBLISHED` 상태로 카탈로그에 노출된다.

[표 22] API 상품 등록 입력 검증 정책

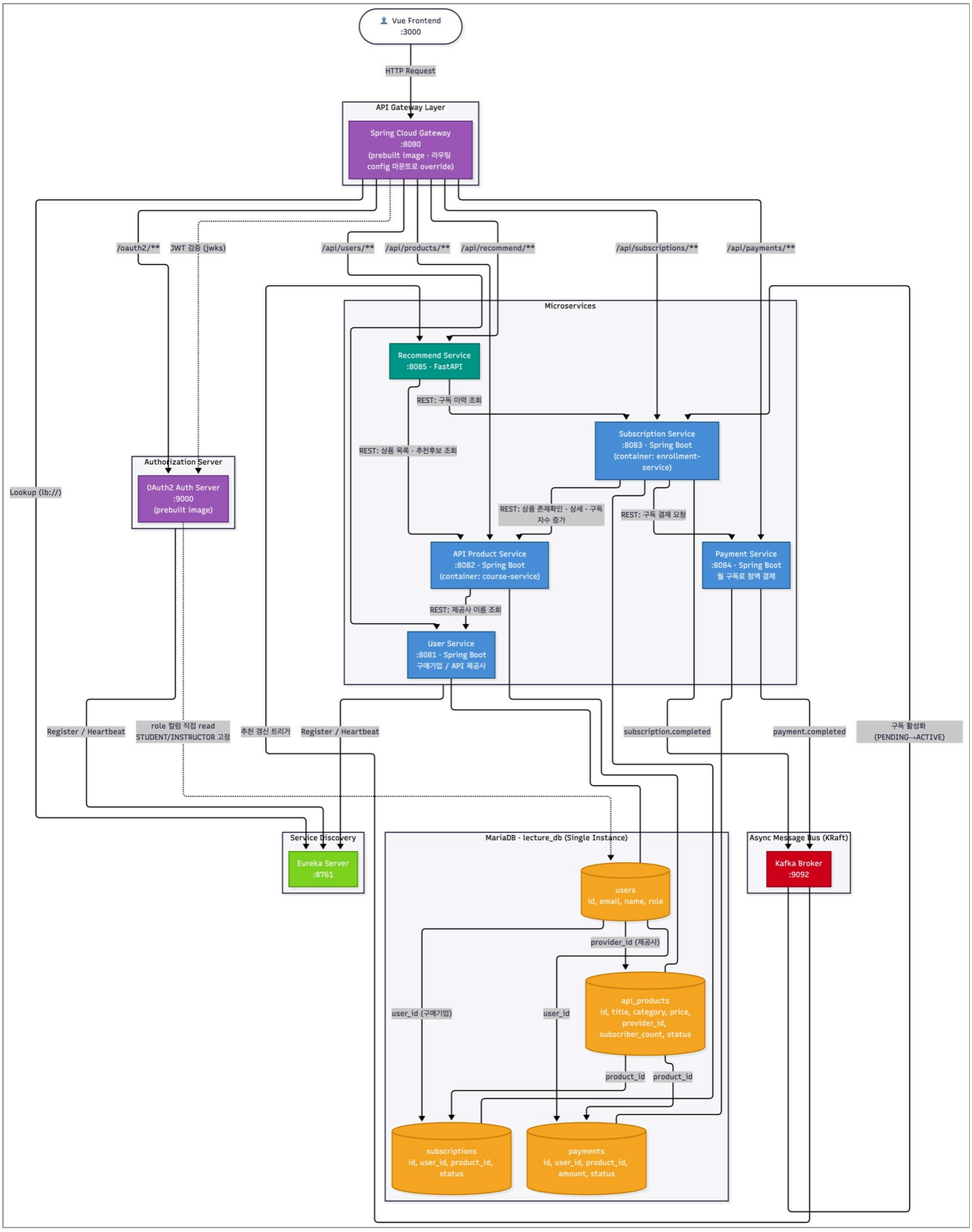
항목	규칙 / 실패 메시지
계정 권한	제공사 계정만 등록 가능 — “제공사 계정만 API를 등록할 수 있습니다.” (`/products/new`는 라우터 `providerOnly` 가드로 구매 기업 접근 차단)
상품명	필수 — “API 상품명을 입력해 주세요.”
설명	필수 — “API 설명을 입력해 주세요.”
카테고리	필수 선택 — “카테고리를 선택해 주세요.”
월 요금	필수, 0 이상의 숫자 — “가격은 0 이상의 숫자로 입력해 주세요.”



[그림 17] 제공사 대시보드 (내가 등록한 API)
등록한 API마다 카테고리·월 요금·구독 기업 수·상품 ID를 카드로 보여주고, 오른쪽에 등록 API 수와 총 구독 기업 수를 요약한다. 구독 기업 수가 곧 인기도 지표이다.

X. 아키텍처 개요

MSA(Microservice Architecture) 기반으로, 제공된 온라인 교육 플랫폼 템플릿 (user, course, enrollment, payment, recommend-service)의 구조를 그대로 재사용하고 도메인만 치환했다.



[그림 18] 아키텍처 구성도

Vue 프론트(:3000) → API Gateway(:8080) → 5개 마이크로서비스(user·product·subscription·payment·recommend). auth-server(:9000)가 JWT를 발급하고 Eureka(:8761)가 서비스 디스커버리를 맡는다. 결제·구독 완료는 Kafka(:9092)로 비동기 전파되고, 단일 MariaDB(lecture_db)를 서비스별 스키마로 나눠 쓴다.

[표 23] 구성 요소

구성 요소	역할
API Gateway (Spring Cloud Gateway)	모든 클라이언트 요청의 단일 진입점
Eureka	서비스 디스커버리
User Service, API Product Service(구 Course), Subscription Service(구 Enrollment), Payment Service	Spring Boot 기반 도메인 서비스
Recommend Service	추천 로직 처리
MariaDB	단일 인스턴스, 서비스별 스키마 분리 (users, api_products, subscriptions, payments 등)
Kafka (KRaft 모드)	서비스 간 이벤트 기반 비동기 처리 (예: 결제 완료 시 구독 활성화 이벤트)
Redis, Docker Compose	로컬 컨테이너 구동 환경

> 서비스 간 상세 호출 및 이벤트 흐름을 표시한 아키텍처 다이어그램은 발표자료(PPT)에 별도로 수록되어 있다.

XI. API 명세 설계 부록 A · 서비스별 도메인 매핑 및 주요 API 명세

Swagger UI에서 확인한 엔드포인트를 기준으로, 기존 템플릿 API를 서비스 도메인에 맞게 재매핑했다. 엔드포인트는 템플릿과 같은 19개이고 새로 추가한 것은 없다. 경로, 필드, Enum 세 가지를 치환해 수강 도메인을 API 마켓플레이스 도메인으로 옮겼다.

A-1부터 A-5까지는 서비스별 매핑표로 전체 엔드포인트를 싣고, 화면 흐름에 직접 쓰이는 11건만 요청과 응답까지 적었다. Sprint 1 범위 6건은 A-6, Sprint 2 범위 5건은 A-7이다. 나머지는 매핑표의 한 줄로 갈음한다. 요청·응답 예시는 실제 호출을 캡처한 것이 아니라 DTO와 시드 데이터를 기준으로 적은 값이다.

A-0. 공통 규약

[표 24] 요청·응답 공통 규약

구분	내용
진입점	API Gateway(8080) 단일 진입. 서비스 포트는 user 8081, product 8082, subscription 8083, payment 8084
인증	`Authorization: Bearer {token}`. auth-server(9000)가 발급한 JWT를 쓴다
사용자 식별	Gateway가 토큰에서 해석해 `X-User-Id` 헤더로 넘긴다. 본문으로 `userId`를 받는 API는 없다
응답 래퍼	`{ success, message, data }`. 성공 시 message는 “성공”
내부 호출	`/internal/` 경로는 래퍼 없이 데이터를 그대로 반환한다
상태 코드	200 조회 · 201 생성 · 400 검증 실패 · 401 토큰 없음 · 404 대상 없음

> 응답 래퍼는 서비스마다 자기 DTO 안에 따로 정의되어 있다. 공용 모듈로 빠져 있지 않은 템플릿 구조를 그대로 두었다.

A-1. user-service (회원 → 구매 기업 / API 제공사)

[표 25] user-service 도메인 매핑

기존 API	변경 후 도메인 동작	구분	비고
`POST /api/users/register`	구매 기업 담당자 및 API 제공사 계정 가입 (role: BUYER / PROVIDER)	사용	A-6-1
`GET /api/users/me`	내 정보 조회. 프론트 권한 판정의 유일한 출처	사용	Sprint 2
`GET /api/users/internal/{id}`	서비스 간 내부 조회 (product-service가 제공사 이름 해석에 사용)	내부	신규 소비자
`GET /api/users/{id}`	사용자 조회 (응답 role이 BUYER / PROVIDER)	미사용	—

> 경로는 바뀌지 않았다. 회원 개념이 그대로 대응되기 때문이고, 바뀐 것은 응답의 role 값뿐이다. `users.role` 컬럼값은 `STUDENT/INSTRUCTOR`로 유지되며 `RoleConverter`가 변환한다. prebuilt auth-server가 같은 테이블을 읽기 때문이다.

A-2. course-service → product-service (과목 → AI API 상품)

[표 26] product-service 도메인 매핑

기존 API	변경 API	도메인 동작	구분
`POST /api/courses`	`POST /api/products`	API 상품 등록 (제공사만). 카테고리, 월 구독료 입력	사용 · A-6-3
`GET /api/courses`	`GET /api/products`	API 카탈로그 전체 목록 (PUBLISHED만)	사용 · A-6-4
`GET /api/courses/{id}`	`GET /api/products/{id}`	API 상품 상세 스펙	사용
`GET .../category/{category}`	`GET .../category/{category}`	카테고리별 탐색	사용 · A-7-1
`GET .../internal/exists/{id}`	`GET .../internal/exists/{id}`	상품 존재 확인 (subscription-service 호출)	내부

기존 API	변경 API	도메인 동작	구분
`GET .../internal/{id}`	`GET .../internal/{id}`	상품 상세 조회 (내 구독 목록 조립용)	내부
`POST .../enrollment-count`	`POST .../subscriber-count`	구독 기업 수 증가 (인기도 지표)	내부 · A-7-5
`GET .../internal/recommend`	`GET .../internal/recommend`	카테고리별 미구독 상품 조회 (recommend-service 호출)	내부

> 필드 매핑은 `instructorId → providerId`, `enrollmentCount → subscriberCount`, `Status.ACTIVE/INACTIVE → PUBLISHED/DEPRECATED`이며 Category는 8종을 1:1로 치환했다. `providerName`은 신규 추가된 필드로, user-service 조회로 해석한다. 목록 조회는 PUBLISHED만 거르고 단건 조회는 거르지 않는다.

A-3. enrollment-service → subscription-service (수강신청 → API 구독)

[표 27] subscription-service 도메인 매핑

기존 API	변경 API	도메인 동작	구분
`POST /api/enrollments`	`POST /api/subscriptions`	API 구독 신청, PENDING 상태로 생성	사용 · A-6-5
`GET /api/enrollments/my`	`GET /api/subscriptions/my`	내 구독 API 목록 (마이페이지)	사용 · A-6-6
`GET .../internal/history/{userId}`	`GET .../internal/history/{userId}`	구독 이력 조회 (recommend-service 호출)	내부
`GET /api/enrollments/user/{userId}`	`GET /api/subscriptions/user/{userId}`	특정 사용자 구독 목록	미사용

> 경로 치환 외에 `courseId → productId`, `activeCourseIds → activeProductIds`, `CourseSummary → ProductSummary`를 함께 바꿨다. 구독 신청 응답은 항상 PENDING이고, 결제 완료 이벤트를 받은 뒤 ACTIVE가 된다.

A-4. payment-service (결제 → 구독 결제)

[표 28] payment-service 도메인 매핑

기존 API	도메인 동작	구분
`POST /api/payments/internal/request`	월 구독료 정액 결제. 결제를 기록하고 `payment.completed`를 발행한다	내부 · A-7-2
`GET /api/payments/user/{userId}`	기업 결제 및 크레딧 사용 내역 조회 (마이페이지 결제 내역)	사용 · A-7-3
`GET /api/payments/{id}`	결제 단건 조회	미사용

> 경로는 그대로 두고 `courseId → productId` 필드만 치환했다. 결제가 완료되면 `transactionId`를 발급하고 `payment.completed`를 발행한다.

A-5. recommend-service (추천)

[표 29] recommend-service 도메인 매핑

기존 API	도메인 동작	구분
`GET /api/recommend/{userId}`	같은 기능군의 대체 및 보완 API 추천 (구독한 건 제외). 최대 5건	사용 · A-7-4
`GET /health`	서비스 헬스체크	사용

> 이 서비스만 Spring이 아닌 FastAPI다. 치환 대상은 경로가 아니라 Kafka 토픽명, 이벤트 키, 내부 호출 경로, ProductCategory 8종이었다. 응답은 공통 래퍼를 쓰지 않고 Pydantic 스키마를 그대로 내보낸다.

A-6. Sprint 1 주요 API 상세 명세

회원가입에서 내 구독 조회까지, Sprint 1 핵심 흐름을 한 바퀴 도는 데 필요한 6건이다. 인증 헤더는 A-0의 공통 규약을 따르므로 여기서는 예시에서 생략했다.

A-6-1. 회원가입 `POST /api/users/register`

인증 불필요 · 치환 없음, role 값만 BUYER / PROVIDER로 바뀌었다

Request

```
{
  "email": "techinno@example.com",
  "password": "password1234",
  "name": "(주)테크이노베이션",
  "role": "BUYER"
}
```

Response 201 Created

```
{
  "success": true,
  "message": "성공",
  "data": {
    "id": 9,
    "email": "techinno@example.com",
    "name": "(주)테크이노베이션",
    "role": "BUYER",
    "createdAt": "2026-08-27T09:41:22"
  }
}
```

400 이메일 형식 오류, 비밀번호 8자 미만, 중복 이메일. 요청은 BUYER / PROVIDER로 받고 DB에는 레거시 값으로 저장된다.

A-6-2. 로그인 토큰 발급 `POST /oauth2/token`

auth-server 담당. OAuth2 표준 경로라 치환 대상이 아니다

Request (x-www-form-urlencoded)

```
grant_type=authorization_code
&code=nT7xK2...
&redirect_uri=http://localhost:3000/callback
```

Response 200 OK

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "openid profile"
}
```

로그아웃은 `/logout`으로 리다이렉트해 auth-server 세션을 끊는다.

A-6-3. API 상품 등록 `POST /api/products`

PROVIDER 전용 · 경로 치환 POST /api/courses → POST /api/products, category Enum 치환

Request

```
{
  "title": "전표·영수증 OCR API",
  "description": "품목·금액·일자를 추출",
  "category": "DOCUMENT",
  "price": 39000
}
```

Response 201 Created

```
{
  "success": true,
  "message": "성공",
  "data": {
    "id": 10,
    "title": "전표·영수증 OCR API",
    "category": "DOCUMENT",
    "price": 39000,
    "providerId": 3,
    "providerName": "SK AX",
    "subscriberCount": 0,
    "status": "PUBLISHED"
  }
}
```

`providerId`는 요청 본문이 아니라 `X-User-Id` 헤더에서 가져온다. 다른 제공사 이름으로 등록할 수 없다.

A-6-4. API 카탈로그 조회 `GET /api/products`

경로 치환 GET /api/courses → GET /api/products · PUBLISHED 상품만 내려온다 (카테고리로 좁히는 건 A-7-1)

Request

```
GET /api/products
(요청 본문 없음)
```

Response 200 OK

```
{
  "success": true,
  "message": "성공",
  "data": [
    {
      "id": 10,
      "title": "전표·영수증 OCR API",
      "category": "DOCUMENT",
      "price": 39000,
      "providerName": "SK AX",
      "subscriberCount": 63,
      "status": "PUBLISHED"
    }
  ]
}
```

목록을 만들 때 `providerId`가 겹치면 user-service를 한 번만 조회한다. 상품 수만큼 내부 호출이 나가지 않게 한 처리다.

A-6-5. API 구독 신청 `POST /api/subscriptions`

경로 치환 POST /api/enrollments → POST /api/subscriptions · 필드 치환 courseId → productId

Request

```
{
  "productId": 10
}
(userId는 X-User-Id 헤더로 전달)
```

Response 201 Created

```
{
  "success": true,
  "message": "성공",
  "data": {
    "id": 1,
    "userId": 9,
    "productId": 10,
    "status": "PENDING",
    "createdAt": "2026-08-27T13:02:41",
    "product": null
  }
}
```

400 존재하지 않는 상품, 400 이미 구독 신청한 상품. 응답 시점의 status는 항상 PENDING이다. 결제 완료 이벤트를 받은 뒤 ACTIVE로 바뀌므로 화면에서는 목록을 다시 조회해야 한다.

A-6-6. 내 구독 API 목록 `GET /api/subscriptions/my`

경로 치환 · 필드 치환 CourseSummary → ProductSummary, instructorName → providerName

Request

```
GET /api/subscriptions/my
(요청 본문 없음)
사용자 식별은 X-User-Id 헤더
```

Response 200 OK

```
{
  "success": true,
  "data": [{
    "id": 1,
    "productId": 10,
    "status": "ACTIVE",
    "product": {
      "id": 10,
      "title": "전표·영수증 OCR API",
      "category": "문서",
      "price": 39000,
      "providerName": "SK AX",
      "subscriberCount": 63
    }
  }]
}
```

`product` 안의 category만 한글 라벨로 내려온다. subscription-service가 화면 표시용으로 바꾸기 때문이고, 카탈로그 API는 Enum 값 `DOCUMENT`를 그대로 쓴다. thumbnail은 템플릿에서 온 필드라 이번 도메인에서는 채우지 않는다.

A-7. Sprint 2 주요 API 상세 명세

카테고리 필터, 결제, 추천으로 이어지는 확장 범위다. 결제와 구독자 수 증가는 화면에서 직접 부르지 않고 구독 신청에 딸려 서비스 사이에서 오간다.

A-7-1. 카테고리별 API 탐색 `GET /api/products/category/{category}`

경로 치환 · 경로 변수가 곧 치환된 Category 값이다 (US-08). category ∈ VISION, SPEECH, LANGUAGE, DOCUMENT, GENERATIVE, FORECAST, ANOMALY, OTHER

Request

```
GET /api/products/category/DOCUMENT
(요청 본문 없음)
```

Response 200 OK

```
{
  "success": true,
  "message": "성공",
  "data": [
    {
      "id": 10,
      "title": "전표·영수증 OCR API",
      "category": "DOCUMENT",
      "price": 39000,
      "providerName": "SK AX",
      "subscriberCount": 63,
      "status": "PUBLISHED"
    }
  ]
}
```

치환 전 값을 그대로 넣으면(예: `/category/BACKEND`) 400이 난다. 목록과 마찬가지로 PUBLISHED만 내려온다.

A-7-2. 구독 결제 요청 `POST /api/payments/internal/request`

subscription-service → payment-service 내부 호출 · 필드 치환 courseId → productId (US-11)

Request (내부 호출, 래퍼 없음)

```
{
  "userId": 9,
  "productId": 10,
  "amount": 99000
}
```

Response 200 OK

```
{ "paymentId": 1, "status": "COMPLETED" }

이어서 payment.completed 발행
{ "paymentId": 1, "userId": 9,
  "productId": 10, "status": "COMPLETED" }
```

subscription-service가 월 구독료를 `amount`로 넘기면 결제를 기록하고 즉시 완료 처리한 뒤, `transactionId`를 붙여 `payment.completed`를 발행한다. 이 이벤트가 구독을 ACTIVE로 바꾸는 신호다.

A-7-3. 결제 및 크레딧 사용 내역 `GET /api/payments/user/{userId}`

마이페이지 결제 내역과 영수증 화면에서 쓴다 · 필드 치환 courseId → productId (US-12)

Request

```
GET /api/payments/user/9
(요청 본문 없음)
응답에 상품명이 없어 화면에서는
productId로 카탈로그를 다시 조회한다
```

Response 200 OK

```
{
  "success": true,
  "message": "성공",
  "data": [
    {
      "paymentId": 1,
      "userId": 9,
      "productId": 10,
      "amount": 99000,
      "status": "COMPLETED",
      "transactionId": "8f14e45f-ceea-...",
      "createdAt": "2026-08-27T13:02:42"
    }
  ]
}
```

A-7-4. 사용 패턴 기반 API 추천 `GET /api/recommend/{userId}`

FastAPI 담당 · 필드 치환 courses → recommendedProducts, Enum 치환 ProductCategory (US-09)

Request

```
GET /api/recommend/9
(요청 본문 없음)
구독 이력의 최빈 카테고리에서
미구독 상품을 구독자 수 순으로
최대 5건 돌려준다
```

Response 200 OK

```
{
  "userId": 9,
  "recommendedProducts": [
    {
      "id": 5,
      "title": "계약서 조항 추출 API",
      "category": "DOCUMENT",
      "price": 75000,
      "providerName": "SK AX",
      "subscriberCount": 27,
      "status": "PUBLISHED"
    }
  ],
  "basedOnCategory": "DOCUMENT",
  "message": "DOCUMENT 카테고리 기반 추천 API 상품입니다"
}
```

이 응답만 공통 래퍼를 쓰지 않는다. FastAPI가 Pydantic 스키마를 그대로 내보내기 때문이고, 프론트도 이 엔드포인트만 `data` 없이 처리한다. 구독 이력이 없으면 전체 인기순 5건을 주고 `basedOnCategory`는 null이 된다.

A-7-5. 구독 기업 수 증가 `POST /api/products/internal/{id}/subscriber-count`

subscription-service → product-service 내부 호출 · 경로 치환 enrollment-count → subscriber-count (US-12)

Request

```
POST /api/products/internal/10/subscriber-count
(요청 본문 없음)
```

Response 200 OK

```
(응답 본문 없음)

구독이 ACTIVE로 바뀔 때 한 번
호출되어 인기도 지표를 1 올린다
```

A-8. 구독 흐름의 호출 순서와 이벤트

구독 신청 한 번에 다섯 서비스가 엮인다. 1~3은 한 요청 안에서 동기로 이어지고, 4 이후는 Kafka로 넘어간다.

[표 30] 구독 신청 시 호출 순서

단계	호출 주체 → 대상	Method · URL 또는 토픽	결과	방식
1	프론트 → subscription	`POST /api/subscriptions`	구독 PENDING 생성	동기
2	subscription → product	`GET .../internal/exists/{id}`	상품 존재 확인	동기
3	subscription → payment	`POST /api/payments/internal/request`	결제 기록 및 완료	동기
4	payment → subscription	`payment.completed`	구독 활성화 트리거	비동기
5	subscription → product	`POST .../internal/{id}/subscriber-count`	구독 기업 수 +1	동기
6	subscription → recommend	`subscription.completed`	추천 갱신 트리거	비동기
7	프론트 → subscription	`GET /api/subscriptions/my`	ACTIVE 확인	동기

[표 31] Kafka 이벤트 본문

토픽	발행 → 구독	본문
`payment.completed`	payment → subscription	`{ paymentId, userId, productId, status }`
`subscription.completed`	subscription → recommend	`{ subscriptionId, userId, productId }`

> 이벤트 DTO의 필드명이 곧 JSON 키다. enrollment-service를 `productId`로 바꾸면서 recommend-service의 `event.get("productId")`도 같이 바꿔야 했다. 한쪽만 바꾸면 예외 없이 None이 되어 원인을 찾기 어렵다.

끝